Appendix A — Aniruddha Singh deep dive — Agentforce for Setup v2 internals

Source: `ANIRUDDHA_SINGH_DEEP_DIVE.md`

CrowdStrike — Aniruddha Singh, Fiercely Technical Round

Salesforce is your primary work — this doc leads with it. ResolveFlow is your portfolio piece; it shows up here only as proof-of-mechanism when a Salesforce claim can't be fully disclosed (Section 7), and in full depth in the companion doc, `RESOLVEFLOW_DEEP_DIVE.md`, if he wants to go deeper on working code than a bridge answer allows.

This doc is built from five real sources, not generic theory: (1) the actual Agentforce for Setup v2 HLD you have access to, (2) your resume, (3) the architecture material you already built in `INTERVIEW_PREP_Hiren_Anajwala_Architecture.md`, (4) a staff-level generic reference on enterprise agent-platform architecture and security (Section 2) that gives you the vendor-neutral vocabulary and mental models underneath everything Section 1 describes for Agentforce specifically, and (5) your own handwritten study notes — woven in throughout Section 1.4, 2.4b–2.4c, 2.5.7b–2.5.9, 2.6.1–2.6.2, 2.7, 2.8, and 2.9, marked by their more granular, whiteboard-ready framing (numbered steps with explicit Input/Output/State/Attack, named attack taxonomies, exact mechanism sketches like the transactional outbox and the idempotency key construction). Where a file already has a clean answer, this doc extends it rather than repeating it — specifically into **scalability**, **distributed-systems mechanics applied to this exact system**, and **harder, more adversarial follow-ups** than an architecture-manager round would ask. Cross-reference `INTERVIEW_PREP_Hiren_Anajwala_Architecture.md` for the parts not repeated here (Parts C, G, H).

---

0. How to run this round

- **Open with Salesforce.** It's your day job, your title says "Lead AI Engineer," and it's what a security company hiring for an agentic-AI role actually wants to probe — production scale, real tenants, real governor limits, not a solo project.
- **Go deep on mechanism before he asks — numbers, not adjectives.** Say "the funnel narrows hundreds of actions to 3–5 candidates via parallel vector search before one grounded LLM call picks among them" instead of "we use RAG for action selection." Section 1 gives you the exact pipeline stages, JSON schemas, and named components to draw from.
- **When a question isn't about Agentforce specifically — it's just "how would you secure/scale a generic agent platform" — answer from Section 2.** That's the vendor-neutral framework; it's where you show you understand the *general* problem, not just your own system's instance of it. The strongest answers move fluidly between the two: state the general principle (Section 2), then show the specific implementation (Section 1).
- **When he pushes on something you can't fully disclose** (exact latency numbers, internal service names beyond what's here, org-scale figures), say so in one sentence and answer at the architecture-pattern level instead.
- **Bridge to ResolveFlow only when useful, not reflexively.** The strongest use of it is: "I can't paste the Salesforce code, but I built the identical pattern end-to-end in a public project — want the full mechanism instead?"
- **Volunteer real limitations before he finds them** — Section 1.7 (parity gaps, latency regression, centralized retrieval scalability) and Section 4.1 (a known consistency gap) give you several, verbatim from the actual HLD's own "Risks" section. Section 2.16's closing litmus-test question — "*if the model is fully compromised*

right now, what's the worst it can actually cause" — is the single best question to ask **yourself** out loud if he asks you to defend any design choice; if your own answer isn't bounded, say so before he catches it.

- **If he goes fully generic-distributed-systems on you** (CAP, rate limiting, sharding, consensus, delivery semantics) — Section 4 gives you the applied version tied to this exact system; `../CrowdStrike_Prep_Deep_Dive.md` §3 has the from-scratch mechanism if he wants you to derive an algorithm live.
- **If he hands you a laptop and says "write something"** — Section 2.13 has six pieces of pseudocode from this exact problem domain (deterministic authz middleware, idempotent execution wrapper, optimistic concurrency guard, ACL-pre-filtered retrieval, tool-selection entropy drift detector, a policy-decision function) — far more relevant here than a random LeetCode problem, and worth having typed out once before Friday so your fingers know the shape.

---

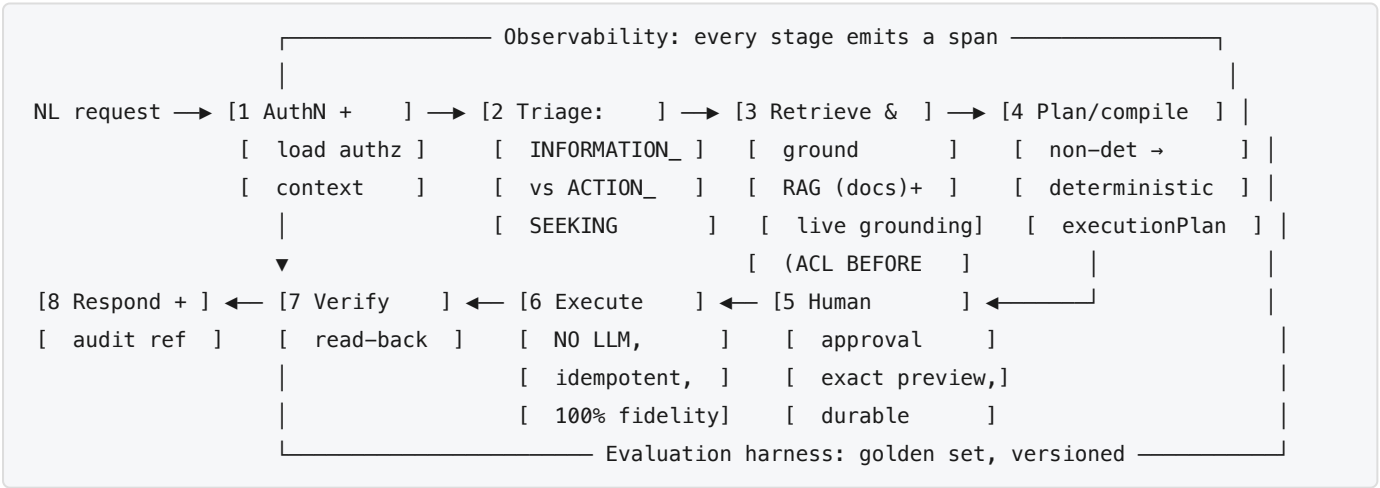
1. Agentforce for Setup v2 — the architecture you own, at full depth

1.1 The one-sentence pitch

> "It's a strategic evolution from a reactive, topic-based v1 — a ReAct planner plus ~45 bespoke Java actions — to a proactive, knowledge-driven system built to solve the **long tail**: the 30–40% of admin use cases that are too numerous to hard-code but represent over 90% of actual user need. The architecture rests on three pillars — a stateful Agent Graph, a multi-layered Knowledge Base, and a set of Generic CRUD Actions — with one non-negotiable design principle: a hard boundary between non-deterministic *thinking* and deterministic *doing*, with a human in the loop before any modification ever executes."

1.2 The reference pipeline — draw this on the whiteboard first

This is the generic shape every specific flow below is an instance of:



[1] AuthN + authorization context — who's asking, what they're allowed to do, *before any model call*. Authorization is derived from caller identity and lives outside the model — it never decides permissions, only proposes.

[2] Triage — cheap, fast classification into exactly two modalities (this is the actual HLD terminology, use it precisely):

- `INFORMATION_SEEKING` — "How do sharing rules work?" / "What is the ID for the 'Sales Team' public group?" — retrieve and understand, org state unchanged.

- `ACTION_SEEKING` — "Clone the user John Smith." — a process that modifies org state, including complex multi-part data queries.

[3] Retrieve & ground — two genuinely different retrieval types, kept separate: conceptual/document RAG (product docs, best-practices — answers "how/why") versus live, tenant-aware grounding against actual org state (answers "what does *this* org look like right now") — **ACL-filtered before retrieval, never after** (Section 5.2 goes deep on why post-filtering is a real leak, not just inefficiency).

[4] Plan/compile — the non-deterministic → deterministic transition. Section 1.3 below is the entire Reasoner mechanism — this is the stage worth the most depth if he picks one to drill into.

[5] Human approval — only for writes/irreversible/low-confidence/policy-sensitive actions; shows the user the **exact resolved plan**, never a paraphrase; implemented as a **durable interrupt** — state persisted to a real store, not in-memory, so the pause survives across separate requests, hours, or a restart.

[6] Execute — contains **no LLM, no reasoning**. Plain deterministic code. Runs each action with 100% fidelity, resolves data dependencies between steps (a `userId` produced in step 1 feeding step 3), and every action is independently idempotent with timeouts and bounded retries, validating its own typed arguments against the caller's authorization again — not trusting that upstream already checked it.

[7] Verify — read-after-write. Don't trust a write API's 200 — query the record back and confirm the actual state changed as expected.

[8] Respond + audit — evidence/citations plus an audit reference (trace ID). Terminates on success, failure, or step/token/cost budget exhaustion — never an unbounded loop.

1.3 Pillar 1 — the Agent Graph: Supervisor & Specialists

Why this shape, not a single ReAct loop: a ReAct loop makes "should I act, and how" a per-turn model decision — the boundary between reasoning and acting is a *behavior* the prompt requests, not a *property* the system guarantees. Splitting into a persistent supervisor plus single-purpose specialists makes each node's job narrow enough to reason about and test in isolation, and — critically — means the one node that can have side effects (the Executor) is structurally incapable of also being the node that decides what those side effects should be.

Node	Role	What it actually does
Conversation Manager (Supervisor)	Persistent home base / master orchestrator	Owns the master state and *all* user-facing communication; delegates to specialists via supervision; **control always returns to the CM after a specialist finishes** — this is what makes the flow resilient and stateful rather than a one-shot dispatch.
Knowledge Node	Authoritative source of truth	Retrieves Product Documentation, every category of NLD, and performs Metadata & API Grounding against the *live* org.
Reasoner Node	Planning / problem-solving intelligence	Translates a goal into a deterministic `executionPlan`. Handles ambiguity, synthesizes across multiple NLDs, runs the full Plan Generation Pipeline (below).
Executor Node	Reliable, predictable execution engine	Takes the finalized `executionPlan`, runs each action call with 100% fidelity, no reasoning, resolves runtime data dependencies.

Triage, precisely: the CM invokes a `TriageRequestIntent` action whose only job is the `INFORMATION_SEEKING` vs `ACTION_SEEKING` classification (1.2, stage 2). If information-seeking, CM supervises the Knowledge Node directly for a synthesized answer. If action-seeking, CM hands the entire planning/execution lifecycle to the Reasoner.

The Reasoner picks one of two paths — this is the "how does it decide fast vs slow" answer:

- **High-Velocity Path** ("express lane") — the goal maps to one self-contained Workflow NLD. The Reasoner is a pure *translator*: read the pre-authored `steps` array, convert directly to an `executionPlan`. Example from

the real spec: "Freeze the user john.doe@example.com" → exact match on `freeze_user.nld` → direct translation, no synthesis needed.

- **Dynamic Synthesis Path** ("deep thinking") — anything more complex than one workflow. Three concrete shapes, all real examples from the design:
- *Multi-NLD merge:* "Clone user X and freeze user Y" → finds the Clone NLD and the Freeze NLD independently, synthesizes them into one two-step plan.
- *Guided Journey / "UBER plan":* "Help me create a new Flexipage" → a Higher-Order NLD describes a multi-stage journey; the CM runs it step-by-step *with* the user, not as one silent compile.
- *Hybrid:* "Clone user X and then debug their login issues" → Workflow NLD drives the clone half, Best-Practices NLDs *dynamically generate* the diagnostic steps for the second half — the plan isn't fully pre-authored, part of it is synthesized live.

The Plan Generation Pipeline — not one LLM call, a formal multi-stage process:

1. **Requirement Synthesis & Clarification** (`GatherMoreDetails` action) — checks the goal against what the docs say is *required*. Missing something ("which user")? Pause via a **Consultative Dialogue Loop**, return questions to the CM, ask the user, loop back.
2. **Scaffold Ingestion** (deterministic, no LLM) — extract the `steps` array from the primary Workflow NLD, or synthesize a scaffold from a Higher-Order NLD. This is the high-level logical blueprint the rest of the pipeline fills in.
3. **Plan Compilation** (`compilePlan` — one action, internally two stages): - **Stage A — Parallel RAG for action discovery:** for *every* scaffold step simultaneously, a fast vector search against the entire indexed action catalog returns the **top 3–5 semantically relevant candidate actions** per step. - **Stage B — Monolithic compile:** one well-grounded LLM call, given requirements + scaffold + every step's pre-filtered candidate list, generates the final JSON `executionPlan` — merging steps and wiring the data flow between them. - **Stage C — Validation & finalization:** parse + schema-validate the JSON; optionally pre-flight-validate against the live org and compute a confidence score.

This funnel is the actual answer to "how does this scale to hundreds of actions" — have it exact: "*A naive approach shows the LLM the entire action catalog every turn — that's slow, expensive, and gets worse linearly as the catalog grows. The funnel converts a hard 'search hundreds of actions' problem into an easy 'choose among 3–5' problem: the NLD scaffold narrows the conceptual space first (for 'Clone User' the agent is already thinking about identifying and creating users, not updating fields), then parallel vector search per step narrows further to a tiny relevant shortlist, and only then does one LLM call reason — over 3–5 options, not hundreds. The catalog can grow to hundreds of actions and the final reasoning step stays exactly as fast and accurate, because the thing that scales with catalog size (the vector search) is cheap and parallel, and the thing that's expensive (the LLM call) never sees the full catalog.*"

The Executor — "the cornerstone of v2's reliability," verbatim from the design's own framing: no LLM, no reasoning logic anywhere in its control flow — plain deterministic code. Accepts the finalized plan and executes each step with **absolute fidelity, no deviation**. Manages runtime state: the output of step 1 (say, a resolved `userId`) becomes the input to step 3 — data-dependency resolution done entirely in code, never re-derived by a model mid-execution.

1.4 Pillar 2 — the Knowledge Base: grounding and ingestion

The knowledge funnel — four layers, high-level to low-level:

Layer	Level	Content
Solutions & Product Documentation	High	help.salesforce.com, architect.salesforce.com, knowledge articles — public, customer-facing; useful for pure information-seeking answers.
Higher-Order NLDs	Medium	*Which direction to take* — internal docs recommending the best feature/approach for a broader goal.

Layer	Level	Content
Workflow NLDs	Medium/Low	*How to build it* — step-by-step breakdown for one specific operation, once direction is chosen.
Best-Practices NLDs / API grounding	Low, per-entity	Extension of API docs — common practices for a given metadata type; the API contract info an action call actually needs.

Natural Language Documents (NLDs) — internal documents describing how to achieve a task *via Salesforce APIs*, written like an engineer's own design docs. A real example worth having memorized (`clone_user.md`):

```
{
  "name": "clone_user.md",
  "entities": ["User", "PermissionSetAssignment", "GroupMembership"],
  "operations": ["Clone", "Clone With Assignment"],
  "permissions": ["Manage Users"],
  "relatedDocuments": ["How to assign a PermissionSetAssignment to a User", "..."],
  "scopes": ["Agentforce for Setup", "Agentforce Vibes"],
  "tools": []
}
```

Prose above the JSON gives the human-readable requirements and the "how to via API" steps (retrieve the source user's ID → create the new User record → if cloning with assignments, retrieve and re-create PermissionSetAssignments and GroupMemberships). **The point of this shape:** the same document is simultaneously human-readable documentation and a machine-parseable contract (entities/operations/permissions the planner grounds against) — that's what "Knowledge as Code" means concretely: adding a new admin capability is authoring one Markdown file like this, not shipping new Java.

NLD Generation Pipeline — how the knowledge base itself scales: a user scenario ("How do I create a permission set?") goes in; an LLM synthesizes a draft from Salesforce Help content, API specs, and existing bespoke-action descriptions; the result is a compliant Markdown + JSON draft; a human (the feature owner) reviews and approves it; only then is it ingested. **This human-curation gate is worth stating explicitly if asked "how do you keep the knowledge base itself from drifting wrong"** — the KB isn't purely LLM-authored-and-trusted; every entry has a human sign-off before it can ground a live plan.

Storage & retrieval, the real sequence — draw this if he asks for the retrieval architecture specifically:

```
Customer org's agent  ──►  QueryDataSource action (prompt + source)
        │
        ▼
  Org CS (Connected App / ECA) — authenticates the org
        │
        ▼
  Custom TokenExchange handler mints an orgJWT
  (scoped to THIS tenant, as an OrgCS/HelpPortal identity)
        │
        ▼
  Retriever call + orgJWT  ──►  SFAP AI Gateway
        │
        ▼
  Gateway routes into Data Cloud (the C360 environment)
        │
        ├──► Unified Knowledge Dataspace (UKD) — Product/Help docs,
        │     same retrieval service Help Agents use, same chunking/
```

```
        │     ranking logic as the core support platform
        │
        └──▶ NLD Search Index — a SEPARATE index, built from
             vectorized DMOs derived from mDLOs (NLDs land in Data
             Cloud via a Metadata Extraction pipeline, not the UKD path)
        │
        ▼
  Chunks returned back through the gateway to the requesting org's planner
```

The tenant-isolation property this buys, stated precisely (this is your strongest security-adjacent line in the whole Salesforce section): "*The `orgJWT` is minted per-tenant by a token-exchange handler before any retrieval call happens — so tenant isolation isn't an application-level filter that could have a bug in it, it's that the downstream retrieval service literally never receives a token that grants access to another org's data. This is the same principle as OAuth token-exchange, RFC 8693 style — swap a caller's token for a narrowly-scoped downstream token before the call that actually touches data.*" Section 2.6 below names the general MCP-adjacent version of this same principle (audience-bound tokens that can't be replayed against a different server).

Ingestion mechanics worth knowing exactly: NLD/Knowledge/API-Grounding content is authored as `.md` in Core; a one-time per-org **Metadata DLO Provisioning** step creates the mDLOs in Data Cloud; then an org-level **Metadata Extraction BT Action** does the actual `.md → mDLO` extraction+ingestion, moving through a real state machine: `New → Processing → Syncing → Data_Copied → Ingestion_Requested → Ingestion_Complete`. (Section 4.1 below uses this exact state machine for the consistency-model question.)

A real, disclosed optimization worth citing: NLD source documents can be served directly from Core's own classpath instead of round-tripping to C360 for every retrieval — cutting one hop out of the latency budget for exactly the content type (NLDs) that doesn't need the UKD's shared-across-orgs freshness guarantees the way product docs do.

1.4a Data Cloud / Data 360 ingestion — stage by stage, with real latency numbers

If he wants the retrieval pipeline at maximum resolution rather than the sequence diagram above, this is the general Data 360 ingestion pipeline underneath it, worth walking stage by stage with actual numbers rather than "it gets indexed eventually":

1. **Source** — knowledge articles, cases, emails, meeting notes, RFP responses, web sources.
2. **Ingestion** — via 360 connectors; runs in both **batch** and **streaming** mode depending on the source.
3. **Transformation/mapping** — data lands in Data Lake Objects (DLOs), then gets mapped to Data Model Objects (DMOs).
4. **Chunking** — breaks content into smaller, semantically meaningful units, saved as a Chunk DMO.
5. **Embedding** — converts each chunk into a numeric representation that captures semantic similarity.
6. **Indexing + ACL metadata** — provisions both a vector index and a keyword index; ACL/permission metadata is captured **at embedding generation/refresh time**, not bolted on afterward.
7. **Query generation** — the retriever is, mechanically, **a wrapper over a SQL query** — it's the bridge between the search index and the prompt template, the same conceptual role a vector-DB SDK plays in a hand-rolled RAG stack.
8. **Retrieval** — the query is vectorized and matched against the index; it returns both the record and a relevance **score**.

Concrete latency/consistency numbers worth citing instead of "eventually consistent" as an adjective:

- CRM ingestion into Data 360: roughly **every 10–15 minutes**.
- Identity resolution: minimum **1–24 hours** — meaningfully slower than raw ingestion, because it depends on enough signal accumulating to resolve identities confidently.

- Search-index filter fields are captured at embedding generation/refresh time — ACL-relevant metadata is only as fresh as the last re-embed.
- Search-index re-vectorization happens on embedding generation/refresh, not continuously.

Data 360 is best described precisely as "a derived, eventually-consistent data plane" — say it in exactly those words if he asks what kind of consistency guarantee it offers; it's not a stronger claim than that, and claiming otherwise is the overclaiming trap Section 2.16 warns about.

The runtime question that actually matters, and the test to apply to it: "*Does this caller have live access right now?*" — for anything where the answer changes the outcome, don't trust the pre-computed index; make a synchronous, live call instead (this is exactly why `TenantGrounding`, 1.5, hits the live org directly rather than the vector index for anything permission-sensitive). **Concrete failure case to have ready:** if access is revoked at 9:00 but a retrieval happens at 9:45 against an index that hasn't re-synced yet, the agent confidently serves **stale, still-indexed data as if it were current** — the system doesn't error, it just quietly answers with the wrong-but-plausible thing, which is the worst failure shape for exactly the "a well-formed, confident answer can be entirely wrong" reason Section 2.9 names as the single most important failure-analysis habit.

Retrieval-time parameters worth being able to name precisely, not just "we tune the retriever": similarity, **top-k** (how many chunks come back), **thresholds** (results below a relevance bar get dropped before they ever reach the prompt), **metadata filters** (filtering on a mutable field — which is exactly why filter freshness matters, per the stale-access case above), and **re-ranking** (fusion ranking across the vector-store and keyword-store results, not just taking the raw vector-similarity order).

1.5 Pillar 3 — Generic Actions (the tool layer)

Four categories, each with a specific job:

1. **Discovery Actions** — locate metadata types/records from an utterance *before* any modification. v2's direction: consolidate many disparate ad hoc search mechanisms into unified actions, eventually backed by a shared **Unified Catalog Metadata Search**. Plus a raw `ExecuteSOQL` escape hatch for when a simple ID lookup isn't enough — the SOQL snippet itself usually comes pre-written from an NLD, not generated freehand by the model.
2. **Context/Grounding Actions** — `QueryDataSource` (replaces v1's `HelpNavigationAction`), `APIGrounding` (the API-contract info a CRUD call needs, sourced from public API docs — the entity must be exposed via Data or Tooling API), `TenantGrounding` (semantic search over the org's own vectorized metadata).
3. **Generic CRUD Actions** — `ExecuteEntityCreate / Read / Update / Delete`. The planner must supply the **API family** (Data vs Tooling) and the **Entity API Name** — this is the piece that *replaces* bespoke per-entity actions (`UpdateUserRecordFields` → `ExecuteEntityUpdate`), and it's the generalization that makes "add a capability = add a Markdown file" true: a new entity type doesn't need a new action implementation, just a new NLD telling the planner which generic action and which entity name to use.
4. **Display Actions** — generic record/list rendering, plus a newer **Generative UI** path (AXL surface) for responses with no pre-built UI: composes a layout from an *allowlist* of existing components (no new LWC is ever generated at runtime), producing a document the client renders — each surface (LEX, Slack, Mobile) renders the same abstract output its own way (Slack maps it to Block Kit, for instance).

A note on the generic-vs-bespoke tension, worth stating explicitly if pushed: Generic CRUD actions are exactly the "capability-scoped generic tool" pattern named in Section 2.6 — they're not an unconstrained "update any field on any record" primitive handed to the model; the planner has to supply a specific API family and entity name, each entity's exposure is explicitly allowlisted via `APIGrounding` (an entity must be *publicly exposed* via Data or Tooling API to be reachable at all), and Salesforce's own field-level/object-level security still applies underneath the generic action at execution time. The genericness is in the *action catalog surface*, not in an unconstrained blast radius.

Scoping the blast radius further, three concrete levers beyond the API-family/entity-name constraint above: a **field allowlist per topic** (a topic can only ever touch the specific fields its NLDs declare, not every field on the entity); **value constraints enforced as policy**, not left to the model's judgment (a generic update action can still refuse an out-of-range or disallowed value deterministically); and **confirmation gated specifically on consequential writes**, not on every generic-CRUD call uniformly — the same risk-tiering idea as 2.5.8, applied at the tool-schema level rather than the pipeline-stage level. Also worth naming precisely: the generic actions are `createRecord` / `updateRecord` / `deleteRecord` / `queryRecord` — genuinely generic operation names — rather than per-entity bespoke names like `createAccountRecord`; the entity itself is always a *parameter*, never baked into the action name, which is the actual mechanism behind "add a capability = author a Markdown file, not new Java" (1.4).

Metadata API vs Tooling API — the distinction he may push on: Metadata API deploys/retrieves *configuration* (objects, flows, profiles), often asynchronous and file-based, built for bulk config changes. Tooling API gives fine-grained, record-level access to metadata for developer tooling — synchronous, SOQL-queryable. The planner has to know which family a given operation belongs to; that's part of what `APIGrounding` supplies.

1.6 How your resume bullets map — say it in exactly this shape

Resume claim	What it actually is, mechanically
"Retrieval-to-action architecture... permission-aware tools execute structured operations"	The Knowledge Node → Reasoner → Executor shape: NL request retrieves versioned NLDs/Skills, live metadata grounds the plan, and the Executor's typed, permission-checked generic actions run it — described in your own words, same pattern.
"Lead architecture... five enterprise AI agent workflows"	Each workflow = a Workflow NLD (or small NLD set) plus the required entity/permission/tool-contract wiring — the content-migration step of the v1→v2 strategy, done five times over real production workflows.
"Onboarded eight enterprise capabilities to Headless 360"	Knowledge Base onboarding + generic-action exposure per capability — diagnosing indexing, routing, permission, and runtime constraints for each one (your Story D below).
"Migrated 5 workflows V1→V2, consolidated 20+ prompts/permissions/tool defs"	Literally the "Content Migration" phase of the documented Migration Strategy: v1 Topics/Instructions → v2 NLDs, bespoke actions kept alive during transition and retired toward generic CRUD.
"Implemented an AI evaluation framework across retrieval quality, tool selection, permission enforcement, structured outputs, failure paths"	Component-level + end-to-end golden-set regression, exactly the "evaluate each component separately AND end-to-end" principle the reference pipeline calls for around the whole loop.
"Championed security-first AI engineering... cutting misconfigured access-management requests ~35%"	Operation-level authorization derived from caller identity, checked independently in code at the Executor boundary — never trusted from the model's proposal.

1.7 Real STAR stories — yours, elevated for a fiercely technical audience

These are already in `INTERVIEW_PREP_Hiren_Anajwala_Architecture.md` Part D — here they're extended with the follow-ups a technical (not architecture-manager) interviewer is more likely to push on, and the "why this lands" framing.

Story A — Action selection didn't scale (the funnel)

S/T: At ~45 bespoke actions, v1's LLM-driven topic selection was already brittle. The roadmap needed hundreds of actions, and "show the model everything" doesn't survive that.

A: Moved to the three-stage funnel (1.3): NLD scaffold narrows the conceptual space → parallel per-step vector search returns 3–5 candidates → one grounded LLM call picks among the shortlist and wires the data flow. The

hard "search" problem becomes an easy "choose" problem.

R: The final reasoning step stays fast and accurate independent of catalog size; new capabilities become a Markdown file, not new Java.

Fiercely-technical follow-ups to have ready:

- "*What's the actual complexity here — walk me through what grows linearly and what doesn't as the catalog goes from 45 to 4,500 actions.*" The vector index size and the per-step search cost grow with the catalog (that's a standard ANN-index scaling problem — HNSW-style, sublinear in practice), but the **expensive part — the LLM call — never sees more than 3–5 candidates per step regardless of catalog size**, because that shortlist is fixed by the top-k parameter, not by the catalog. So token cost and latency on the compile call are decoupled from catalog growth; only retrieval-index cost and (very cheap) vector-search latency scale with it.
- "*What happens when the top-5 candidates for a step are all wrong — a genuinely new action type the index has no good match for?*" This is a retrieval-recall problem identical in kind to what you handle in RAG generally: if recall@5 misses the right action, the compile call can only pick the best of a bad set, or (if the design allows it) decline and escalate for clarification. The mitigation is the same discipline as `groundedness_ok` in your ResolveFlow work — measure recall on a real eval set per action-catalog size, don't assume the funnel just works because it's a good idea.
- "*Why parallel vector search per step instead of one big search over the whole scaffold at once?*" Because a scaffold step like "identify the user" and a step like "create the permission set assignment" need *semantically different* candidate actions — merging them into one search would dilute the query and risk cross-step contamination in the shortlist. Per-step search keeps each search's query focused on exactly what that step needs, and running them in parallel (not serially) is what keeps the latency cost of doing N separate searches roughly flat instead of N×.

Story B — Migrating 5 production workflows V1→V2 without regressions

S/T: Five production access-management workflows on the legacy retrieval-driven architecture had to move to the redesigned platform — consolidated prompts, permissions, tool contracts, structured outputs — with zero user-visible regression.

A: Built utterance matrices / golden datasets **per workflow**, covering Skill retrieval, input collection, tool selection, API execution, and response rendering — run as regression on every change. Used the documented co-existence strategy: v2 behind a permission gate alongside v1, content migrated (Topics/Instructions → NLDs), bespoke actions kept alive during the transition and retired gradually toward generic CRUD.

R: Consolidated 20+ prompts/permissions/tool defs/execution paths; guidance reused by 10+ teams.

Fiercely-technical follow-ups:

- "*How do you know 'no regression' isn't just 'no regression on the cases you thought to test'?*" Real, honest answer: you don't get a proof, you get a calibrated confidence — the golden set is seeded from real production traffic and known edge cases, not hand-imagined ones, and it's a *living* set that grows every time a real production failure is found (this is the same "replay production failures" discipline from your eval-framework resume bullet). The permission-gated co-existence is the actual safety net underneath the eval set: even a regression the golden set missed is caught by a small blast radius (gated rollout) instead of an all-at-once cutover.
- "*What's the actual cutover mechanism — big-bang or gradual, and how do you decide when it's safe to flip a workflow fully to v2?*" Gradual, gated, per-workflow, not all five at once — a workflow moves off v1 only once its own golden set passes and (ideally) a canary cohort has run clean in production for a bake period. Treat each of the five workflows as an independent rollout unit, not one atomic migration — that's what keeps a regression in one workflow from blocking or contaminating the other four. This is a direct instance of Section

2.10's "five independently-versioned change axes" principle: content (NLDs), tool defs, and permissions all moved here, and each needed its own version/rollback story, not one bundled release.

Story C — Production trust failure, root-caused via Splunk

S/T: An agent workflow produced wrong or low-trust output in production — root cause spans a genuinely distributed boundary: agent orchestration ↔ metadata grounding ↔ APIs ↔ caching ↔ authorization.

A: Reproduced the issue, formed hypotheses, traced across system boundaries in Splunk to isolate **which layer** actually failed — knowledge/retrieval, reasoning, presentation, grounding staleness, a cache-key bug, or a permission issue — fixed the root cause, then added both a regression case and an eval scenario so the same failure mode is caught automatically going forward.

R: Contained; the specific failure mode is now baked into the golden set so it can't silently return.

Fiercely-technical follow-ups:

- "*Walk me through your actual debugging method — how do you isolate 'which layer' in a system with this many moving parts, concretely, not just 'I traced it in Splunk'?*" The method is elimination by boundary, in order of cheapest-to-check first: (1) was the retrieved grounding actually correct and fresh — check the retrieved chunk/metadata against the live org state directly, bypassing the agent entirely; (2) if grounding was right, was the compiled plan correct given that grounding — inspect the `executionPlan` JSON itself, which is a deterministic artifact you can diff against what *should* have been generated; (3) if the plan was right, did the Executor run it with fidelity — check the actual API calls made against the plan, verbatim; (4) if execution was right, was the *response* rendering or summarizing it wrong. Each of those four checks is against a **deterministic artifact** (the retrieved doc, the plan JSON, the API call log, the final render) — you're never trying to debug "what the model was thinking," you're diffing concrete intermediate outputs against expectation, stage by stage. This is exactly why the pipeline emits a span per stage (1.2) — without that, this kind of isolation is guesswork. It's also the concrete, applied version of Section 2.9's failure-mode table and Section 2.10's "the shape of the trace is itself a signal" principle.
- "*Give me a concrete example of a caching bug in this kind of system, not the abstract version.*" A stale-grounding bug: a customer changes their org's sharing rules, then immediately asks the agent to act based on current rules — if the Knowledge Node's live-grounding query hit a cache keyed only on the query text (not on org state / a metadata version) it could serve pre-change data, and the agent would confidently act on stale reality. The fix pattern: cache keys for anything org-state-dependent must include something that changes when the underlying state changes (a metadata version/ETag, not just wall-clock TTL), or skip caching entirely for the specific query classes where staleness is unacceptable — this is exactly Section 2.11's semantic-cache warning (a cache keyed only on query text can leak across principals/states that should be distinguished) applied to staleness instead of tenant leakage, same root mistake.

Story D — Headless 360: a capability that wouldn't onboard cleanly

S/T: Onboarding 8 enterprise capabilities to Headless 360; some didn't expose cleanly — indexing, routing, permission, or runtime constraints blocked them.

A: Worked each boundary individually — API exposure, tool discovery, metadata grounding, reusable Skills, MCP execution. Where a constraint genuinely couldn't be fixed in scope, documented the boundary and the migration path explicitly rather than forcing a bad workaround.

R: 8 capabilities onboarded; integration guidance adopted by 10+ internal teams; misconfigured requests down ~35%.

Fiercely-technical follow-up:

- "*Give me one specific example of a constraint you couldn't fix in scope, and what you actually did about it.*" Have one concrete, honest example ready — even a disguised/generalized one is fine if the real specifics are confidential. The shape that lands well: "a capability's underlying entity wasn't exposed via Data or Tooling API at all, which the generic CRUD actions require by design — rather than building a bespoke workaround action that would defeat the whole point of the generic-action migration, I documented it as an explicit platform gap with the owning team and scoped the capability out of that onboarding wave rather than compromising the pattern for one exception." **The point to land:** knowing when *not* to solve a problem locally — because the generalizable fix belongs to a different team or a different layer — is itself the architectural judgment being tested here.

Story E — Prompt injection / retrieved-instruction defense

S/T: RAG over documents *and* live org data means retrieved content is, in principle, attacker-controllable — a malicious or compromised data source could contain text shaped to look like an instruction.

A: Treat retrieved content as **untrusted data, never as instructions**, as a trust-boundary property, not a prompt-wording fix: the tool allowlist a plan can select from is derived from the *caller's* authorization, never from anything retrieved; arguments are validated independently in code against that same authorization; there's an independent policy layer between "model proposes" and "code executes"; high-impact actions get a human approval gate regardless of model confidence; retrieved content/tool calls/results are redacted-and-audited so an injection attempt is forensically visible even when it fails; adversarial cases live permanently in the eval set so a fix can't silently regress.

R: Injection is treated as a trust-boundary architecture problem, not something you try to patch away at the prompt level.

Fiercely-technical follow-up — his likely next question:

- "*Concretely, where in THIS architecture — Agentforce v2 specifically — does that trust boundary actually live?*" At the Executor. The Reasoner can be fed poisoned retrieved content and, worst case, compile a plan that *proposes* something wrong — but the Executor only ever runs a `executionPlan` whose every action call is checked against an **operation allowlist** (Generic CRUD actions plus a fixed, vetted set of bespoke ones — there is no "arbitrary action" the plan can invoke) and whose arguments are validated against the caller's actual permission set, independent of what the plan says. And for any write, the human approval gate (stage 5) shows the **exact resolved plan** before it ever reaches the Executor — so even a fully successful injection that fools the Reasoner into proposing something malicious still has to survive a human looking at the literal action it's about to take. This is the identical defense-in-depth shape as ResolveFlow's `execute()` refusing to run without an independently-checked approval flag (companion doc, Section 1.4/2.5) — same principle, same reason: the thing that actually prevents an unauthorized write is structural, not "the model was told not to." It is, precisely, Section 2.5.5's distinction: the Reasoner's honesty is a **soft guardrail** (behavioral, can in principle be fooled); the Executor's allowlist + independent authz + human approval are **hard guardrails** (deterministic, the model cannot argue its way past them). Say that framing explicitly — it's the single sharpest line in this document.

---

2. The Generic Trust-Boundary Model — the framework underneath everything above

This is vendor-neutral. Reach for it whenever a question isn't really about Agentforce specifically — it's "how would you build/secure/scale an enterprise agent platform in general." Move fluidly between this section (the general principle) and Section 1 (Agentforce's specific instance of it) — that fluency is itself a signal of depth a canned, single-system answer can't fake.

2.1 The core problem, stated precisely

> Enterprise agent platforms let a free-text request drive real side effects on business systems. The component that decides what to do — an LLM — is **non-deterministic, cannot be exhaustively tested, and is influenceable by anything in its input**, including data that was never meant to be an instruction. Previously "what the system does" was a fixed, auditable code path authored by engineers; now it's *synthesized at runtime* from a prompt, a tool catalogue, and retrieved context — and the same input can legitimately produce different outputs on different runs.

The engineering problem this creates: how do you let a probabilistic, untrusted reasoning component *propose* actions, while every actual effect on real data is governed by deterministic, testable, fail-closed controls that don't depend on the model behaving correctly?

Name the lineage precisely if asked "is this a new problem": it generalizes the **confused deputy problem** (Norm Hardy, 1988) — a component holding more authority than the party invoking it can be tricked into misusing that authority. An LLM-driven agent is a particularly potent instance, because the "confusion" can now be induced through natural language, not just a crafted API call (full treatment in 2.5.3).

2.2 The layered architecture — trust zones, not services

```
[ End User / Channel ]
        |
[ Agent Runtime: intent classification, planning, orchestration ]
        |
[ Policy / Trust Boundary Layer: input & output filtering, masking, injection defense ]
        |
[ Model Provider: the LLM itself ]
        |
[ Tool / Action Execution Gateway: schema validation, authorization, rate limiting ]
        |
[ Systems of Record: transactional data, search/retrieval indexes, external APIs ]
        |
[ Human-in-the-loop approval, where required ]
        |
[ Audit / Observability substrate, cutting across every layer ]
```

Key properties, stated as answers you can give directly:

- **Layered decomposition:** experience/channel layer → agent-definition layer (mostly declarative config: topics, instructions, available actions) → runtime/orchestration layer (the planner loop) → tool/execution layer (the code that actually touches data).
- **The reasoning plane is untrusted; the control plane is trusted.** The single most important structural decision in any such system is a hard boundary between the component that *decides* and the component that *authorizes and executes*. Everything crossing this boundary is **data**, never **authority**. (This is exactly Section 1.3's Reasoner/Executor split — the general principle, instantiated.)
- **Grounding/retrieval is a first-class subsystem**, not an afterthought — its own pipeline with its own latency, staleness, and access-control characteristics.
- **A standardized tool-calling protocol** (the kind popularized by vendor-neutral specs for connecting models to external tools — MCP is the concrete instance you know) decouples tool providers from agent runtimes, but introduces its own trust boundary at every server connection (2.6).

2.3 Component responsibilities

Generic role	Responsibility	Salesforce analog
Agent Runtime	Plan → act → observe loop per turn; classifies intent; decides which tool to call and with what arguments. Output is always a *proposal*, never an authorized effect.	ConversationManager + Reasoner (1.3)
Policy Engine (Guardrail/Trust Layer)	Platform-level checkpoint every model interaction passes through: masks sensitive data, screens unsafe content, screens injected instructions, produces an audit record. **Fails closed if unavailable.**	The independent policy layer sitting between plan and execution (Story E)
Retrieval Service (Grounding Pipeline)	Ingests, chunks, embeds, indexes; serves permission-filtered context so the model reasons over real facts.	Knowledge Node + UKD/NLD indexes (1.4)
Tool Gateway (Execution Gateway)	The single choke point where a proposed call becomes a real effect: schema validation, entity resolution, re-derived authorization, rate limits, idempotency, approval routing, execution, audit write.	Executor Node (1.3)
Authorization Layer	Deterministically answers "does this identity have the right to do this, to this data, right now?" — independent of anything the model said.	Permission checks at the Executor boundary, backed by Salesforce's own object/field/record security
Human-in-the-Loop Service	Durable workflow routing high-consequence proposals to a human, showing the *actual* validated arguments, blocking execution until a decision is recorded.	Pipeline stage 5 (1.2)
Idempotency/ Concurrency Store	Tracks idempotency keys and version tokens so retries can't double-apply an effect and concurrent writers can't clobber each other.	Discussed applied in 4.2
Audit/Observability Substrate	Immutable, correlated record of every *decision* (not just every outcome).	Trace ID + audit reference, pipeline stage 8
Tool/Action Definitions	Typed contracts (name, description, parameter schema) — a supply-chain artifact, reviewed/versioned/pinned like any dependency.	NLDs + Generic Action schemas (1.4/1.5)

2.4 The fully generic 14-step request/data flow

Draw this if he wants the vendor-neutral version rather than Agentforce's specific 8-stage pipeline (1.2):

1. **Ingress and authentication** — utterance arrives over an authenticated session.
2. **Guardrail scan (input)** — screen raw utterance for injection patterns before it enters any prompt.
3. **Intent classification/planning (turn 1)** — bounded task scope selected.
4. **Grounding retrieval** — scoped to **the requesting user's own access**, not the agent's; returns permission-filtered chunks.
5. **Guardrail scan (context assembly)** — retrieved content inserted as clearly delimited, non-authoritative *data*.
6. **Tool selection (turn 2)** — model emits a structured proposal: tool name + arguments. A *request*, not an executed action.
7. **Schema and semantic validation** — reject malformed input; check the proposal makes sense against current world state (target still exists, subject still active).
8. **Authorization (independent re-derivation)** — checked using the **real caller's identity**, never the model's claim.
9. **Policy decision** — permit / deny / require-approval.
10. **Human approval (conditional)** — human sees the actual arguments, not a paraphrase.
11. **Idempotent execution** — exactly once per idempotency key, under the caller's own authority wherever possible.

12. **Output guardrail scan** — screen the result before it re-enters the model's context for response composition (masking, leakage checks).
13. **Response composition (turn 3)** — model summarizes the outcome.
14. **Audit write** — every step above, not just the final outcome, written to an immutable, trace-correlated log.

The critical invariant, say it exactly this way: "*Authorization is re-checked at execution time, against live state and the real caller's identity — never inferred from what was decided at planning time.*"

2.4b An even more granular walkthrough — Input/Output/State/Attack per step

If he wants you to whiteboard a request end-to-end at maximum resolution rather than the 14-stage vendor-neutral flow above, this is the same journey broken into 21 steps, each with its **Input**, **Output**, **State**, and (where relevant) its **Attack** named explicitly — this is the format that reads as "I've actually built this," because a generic answer doesn't usually carry a named attack surface per step.

1. **User input** — a channel surface (in-app chat, service console, programmatic caller). Security: the channel's own transport (TLS); prompt injection can be embedded directly in the utterance; a channel outage is itself a failure mode worth designing for.
2. **Authentication** — Input: credentials/token. Output: an authenticated principal. State: session/token — deterministic. Attacks: token theft, an over-privileged connected app, confused deputy. Retry path: re-run OAuth, don't silently degrade.
3. **Session** — the agent runtime's own session manager. Output: a session handle holding conversation state, deterministic. Attack: session fixation or token leakage.
4. **Agent runtime & configuration load** — the runtime loads the agent's own configuration. Input: agent ID. Output: an in-memory agent definition. **The configuration itself is a security boundary** — worth saying explicitly if asked "where does authorization actually start": it starts before the first model call, at what configuration the runtime is even allowed to load for this caller.
5. **Context assembly** — Input: session + principal. Output: a context bundle.
6. **Topic/intent selection** — the reasoning engine classifies the request against a topic (inputs: utterance, context, topic definitions). Output: the selected topic.
7. **Instructions** — topic instruction + context, merged. Output: the augmented reasoning prompt.
8. **Grounding** — what the plan needs. Output: grounded, retrieved data merged into context. **Security boundary: the retrieval ACL boundary** — what gets fetched is itself plan-influenced, which is exactly why it has to be filtered before retrieval, not after (Section 5.2).
9. **Retrieval** — the step that actually derives the correctness of everything after it; a bad retrieval here corrupts every downstream stage even if every later stage behaves perfectly.
10. **Tool/action discovery** — a topic-scoped action catalog, not the full catalog (this is the funnel principle from 1.3, generalized).
11. **Action selection** — the planner selects an action plus its arguments.
12. **Parameter generation** — the model emits typed arguments — concretely, something like a set of user IDs and permission-set IDs, never free text.
13. **Schema validation** — reject malformed input cheaply, before it's trusted for anything downstream.
14. **Authorization** — grounded in the platform's own action/permission model, not the model's say-so. Attack: confused deputy — this is the step it's actually aimed at.
15. **Confirmation** — a deterministic gate in front of execution, not a model decision.
16. **Execution.**
17. **Downstream API/data call** — the actual persistence layer (Salesforce API/Data, in the Agentforce case).
18. **Result & observation** — the runtime captures what was actually executed, not what was merely planned.
19. **Re-planning** — if the result changes what's needed next.
20. **De-masking for display** — anything masked going into the model gets selectively unmasked for the human-facing response, under its own policy.
21. **Audit logs** — and everything upstream of this, not just the final outcome.

What the trust layer specifically does, stated as three concrete jobs — have this ready as its own self-contained answer: a) **Data masking of PII** — before the prompt ever reaches the model/token, not after. b) **Zero data retention** at any third-party model provider the prompt passes through. c) **Prompt defense and audit trail** — the same defense-in-depth discussion as Section 2.5.4, tied to a concrete enforcement point in the pipeline rather than floated in the abstract.

A shorter, compressed sketch of the same journey if he wants something faster to draw: `Authenticate → Context → ClassifyIntent → Grounding → RiskClassify → Plan ⇄ ValidatePlan (replan loop) → Execute → Observe`, with a second branch off `Plan` for `Needs more info → Replan → ProposeWrite → Authorize → Execute → Verify → Audit → Respond`. The only stage this compressed version names that the 21-step and 14-step versions leave implicit is **RiskClassify**, sitting right after Grounding and before Plan — worth drawing as its own explicit box if he wants to see exactly where risk tiering (2.5.8) plugs into the pipeline, rather than treating it as a property of the approval stage alone.

2.4c The agent execution context as four boundaries — where authority actually changes hands

A tighter way to answer "where exactly can this go wrong" than walking the full 21-step list: name the four handoffs where authority crosses a boundary, because each one is a place a confused-deputy or over-authorization bug can hide.

```
Human → resolves running user → Agent Runtime → (prompt + tool schema) → Planner
→ Tool Args → Tool → DB
```

1. **User → Agent** — the running user is decided once, at the start, and is **rarely re-decided** for the rest of the interaction. If the interaction spans a long session, ask explicitly whether re-authentication or re-derivation ever happens, or whether the original decision is trusted indefinitely.
2. **Planner → Router** — there is **no re-authentication** at this hop by default; the router trusts that the planner's caller context is still the same one established at step 1.
3. **Router → Tool** — this is the hop that actually converts **end-user identity into system authority** — the point 2.5.3's strongest mitigation (execute under the caller's own authority) has to hold at.
4. **Tool → DB** — the hop most likely to be assumed-safe and actually isn't: **CRUD and field-level-security are not automatically enforced** just because the tool layer checked something upstream — the database call itself has to independently re-check, or this is exactly where a "we validated it in the tool" claim quietly stops being true.

The question this framework is built to answer fast: given any specific bug report or hypothetical, name *which of the four boundaries* actually failed — it's a much sharper answer than "authorization broke somewhere."

2.5 Security model

2.5.1 Threat framing — name the frameworks

Model every trust-boundary crossing (user→channel, channel→runtime, runtime→model provider, runtime→tool gateway, gateway→data store, gateway→external tool/server, runtime→human approver, retrieval-store→runtime) against STRIDE (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege). Name two public, vendor-neutral bodies of work worth citing by name in any system-design discussion: **the OWASP Top 10 for LLM Applications** (Prompt Injection, Excessive Agency, Sensitive Information Disclosure, Supply Chain, Unbounded Consumption) and **MITRE's ATT&CK-style catalog for AI systems** (concrete attacker techniques against ML/LLM systems). Both are checklists and shared vocabulary — neither tells you *where* in your architecture to put a control, or whether it's deterministic. That's still an architecture job. **The recurring finding across all boundary crossings:** the highest-leverage controls are (1) always executing under the real caller's

authorization context, (2) treating every inbound block of text — user input, retrieved documents, tool output alike — as untrusted, and (3) failing closed whenever a security-relevant dependency is unavailable.

2.5.2 Authentication vs. authorization

Different questions, never conflate them. **Authentication** proves who is making a request. **Authorization** decides whether that identity is entitled to a specific effect on specific data, right now. A system that treats "the user is logged in" as sufficient to let an agent act broadly on their behalf has confused the two — exactly the gap that produces confused-deputy vulnerabilities.

2.5.3 The confused deputy problem — full definition, and mitigations in strength order

Coined by Norm Hardy, 1988: a program with more authority than the party who invoked it can be manipulated into misusing that authority on the invoker's behalf. An AI agent is a near-perfect instance: it often runs under a service identity or elevated integration credential with broad reach, while individual end users have much narrower rights. If the agent doesn't re-check the *end user's* authority before acting, the user can reach data or effects through the agent they could never reach directly — and a prompt injection makes this exploitable by a third party, not just a curious end user.

Mitigations, in order of strength:

1. **Execute under the caller's own authority wherever the caller has a downstream identity** — the single strongest fix, because it makes the agent's blast radius mathematically no larger than what the human could already do. (This is exactly the orgJWT-per-tenant mechanism in 1.4, and exactly ResolveFlow's per-visitor OAuth token — same fix, two systems.)
2. Where elevation is unavoidable (a background lookup against a system the user has no direct access to), use a narrowly-scoped, separately-audited service credential rather than a broad standing one, and re-verify the user's authority for anything sensitive derived from that lookup.
3. Never let the model's own assertions ("the user is an admin," "policy allows this") substitute for a real permission check.
4. Bind any delegated token to the specific resource/audience it's meant for, and never forward a token upstream to another service unmodified — otherwise that service inherits whatever the original token could reach.

A concrete worked example, worth walking through by name if he wants a story instead of the abstract definition: "*Dana* (the end user) → *the agent runtime and execution layer (the deputy)* → *the deputy's own special authority (an integration or service-account permission, broader than Dana's)* → *a protected resource Dana shouldn't reach on her own. Dana's only lever into this chain is the prompt itself — the user-controlled request. If the deputy ever uses its* own *authority to satisfy that request without re-checking whether Dana specifically is entitled to the result, that's the vulnerability, full stop — it doesn't require a clever exploit, just a system that forgot to recouple authority to the actual requester.*" Defense: recouple authority to the requester and recheck it at the point that matters — concretely, every hop in `Dana → Agent → Tool` re-derives an authorization check against Dana, never against the agent's own standing credential, before the data source is ever touched.

2.5.4 Prompt injection — direct vs. indirect, precisely

Direct injection: the user's own input tries to override system instructions ("ignore your rules and…"). **Indirect (stored) injection:** instructions are smuggled in through content the agent *reads* rather than through the conversation — a poisoned document in a retrieval index, a malicious field value in a record, an adversarial tool description, or text embedded in an external tool's response. **Indirect injection is more dangerous operationally** because the end user may be entirely innocent — the attacker's payload rides in through a "trusted" data path. (This is precisely Story E's scenario — a compromised or malicious data source, not the conversation itself.)

Defense is **layered, never a single fix**: input scanning; strict separation of "instructions" from "data" in prompt assembly; treating all retrieved/tool-originated text as non-authoritative; constrained/typed tool schemas that bound what a manipulated plan can actually do; deterministic authorization indifferent to what the model "decided"; egress controls on where output can go. **No layer alone is sufficient, and no combination claims to make injection provably impossible** — the goal is to raise the cost of a successful attack and bound the damage of one that partially succeeds, not guarantee immunity. Say that honestly if pushed — overclaiming "we've solved prompt injection" is a tell.

2.5.5 Hard guardrails vs. soft guardrails — your single best line

Soft guardrails are behavioral: system-prompt instructions telling the model what not to do. The model *may* comply, and can be talked out of compliance via injection, jailbreak phrasing, or plain misgeneralization. **Hard guardrails** are deterministic: code-level checks the model cannot argue with — schema validation, authorization checks, allowlists, rate limits, human-approval gates. **Every security-relevant guarantee must rest on a hard guardrail.** Soft guardrails are fine for quality/nuisance reduction, never for anything where a bypass has real consequences.

The test to apply to any claimed control, verbatim, memorize this: "*Can the model talk its way past this?*" If yes, it's soft, and cannot be the last line of defense. This is the exact question underneath Story E's answer and underneath why ResolveFlow's `execute()` re-checks `approved` in the function body instead of trusting graph routing — both are the same move: convert a soft-guardrail-shaped risk into a hard-guardrail-shaped guarantee.

2.5.6 Secure tool/action design

A tool definition is not documentation — it's the interface the model uses to decide what to call and how, and is therefore *part of the security boundary*.

- **Prefer narrow, verb-specific actions** ("revoke this specific grant") over generic, broadly-parameterized ones ("update any field on any record"). A narrow tool is easy to authorize, easy to audit, and has a provable blast radius; a generic tool hands the model a capability whose actual reach is bounded only by whatever policy code you can write to constrain it — any gap is directly exploitable.
- **Validate structure, then meaning, then effect.** Reject malformed input cheaply (schema); check the request makes sense given current state (semantics); authorize; execute. Each layer narrows what the next has to trust.
- **Treat every optional parameter, relaxed "required" flag, or broadened enum as an API-contract *and* security-boundary change** — it can silently widen the set of records/effects reachable through a tool even when nothing in the backing authorization config changed. Any tool-schema change should trigger a full security-relevant regression re-run, not just a functional smoke test.
- **Never expose a raw/low-level query or update capability to the model.** Every exposed capability should be a bounded, purpose-built operation. (Note the tension with Agentforce v2's own `ExecuteSOQL` escape hatch, 1.5 — its mitigation is that the SOQL snippet is pre-written in an NLD, not freely generated by the model at runtime — that's the specific design choice that keeps it from violating this principle. Good to be ready to make that connection explicitly if he probes it.)

2.5.7 The execution gateway pattern

A single, non-bypassable choke point every proposed tool call must pass through before any real-world effect: schema validation → semantic/entity-resolution validation → authorization (re-derived from the real caller) → rate limiting → idempotency-key check → human-approval routing (if required) → execution under the caller's authority → output normalization → audit write. **The value:** centralizes every security-relevant check in one reviewable place, rather than trusting each individual tool's implementation to get all of them right independently. (Exactly the Executor Node, 1.3 — say so explicitly.)

2.5.7b The gateway as eight gates, sequentially — a tighter mnemonic

If asked to enumerate the gateway's checks by name rather than describe it as one blob, this ordering is worth having memorized exactly — it's the same execution gateway pattern above, broken into gates you can count on your fingers:

1. **Tool allowlist** — is this tool even allowed for this user?
2. **Argument schema validation.**
3. **Decide whose permissions matter** — always the real caller's, never the model's claim about who's asking.
4. **Execute the authorization decision** — deterministic, code-level.
5. **Mint a capability** — a short-lived, narrowly-scoped token for this one operation, not a standing credential.
6. **Egress control** — confirm secure/sensitive fields aren't about to leave the boundary unmasked.
7. **Tamper-evident audit** — record what happened, immutably, before or alongside the effect itself.
8. **Confirmation on consequential writes** — the human-approval gate (2.5.8), for the tier of action that needs it.

The layered-bounding line worth having verbatim (a sharp way to describe why no single gate has to be perfect on its own): "*The topic bounds which actions are even visible; the schema bounds what each action can express; and the backing code, running as the authenticated user, bounds what can actually be executed. Three independent, narrowing constraints — not one gate that has to catch everything.*"

Seven hard controls, if he wants a shorter enumerable checklist instead of eight sequential gates: authentication → authorization (CRUD/FLS/sharing policy) → schema validation (oversized or malformed args rejected) → API controls (tool allowlisting) → transaction constraints (DB integrity, atomicity, referential rules) → confirmation gate → network restrictions. Same set of ideas as the eight gates above, organized as a punch list instead of a pipeline — use whichever framing matches how the question was actually asked.

2.5.8 Human-in-the-loop and risk tiering

Autonomy should decrease as consequence increases:

Tier	Example	Typical gate
T0 — Read-only	Look up a record	Fully automatic
T1 — Reversible, low-impact write	Update a status field	Automatic, logged
T2 — Consequential write	Issue a moderate refund	Confirmation required
T3 — High-value / hard-to-reverse	Grant a sensitive permission	Human approval, distinct approver from requester
T4 — Bulk / irreversible	Mass deletion, bulk permission change	Approval + rate limiting + hard caps

Two failure modes to design against explicitly:

- **Approval fatigue** — if everything requires approval, humans rubber-stamp and the gate becomes decorative. Fix: risk-based routing so only genuinely high-risk proposals reach a human, and always show a real diff, never a paraphrase.
- **Treating human approval as *replacing* authorization rather than *complementing* it** — a human clicking "approve" on an unauthorized action does not make it authorized. (Say this if he pushes on "isn't a human in the loop enough" — no, because the human is approving a *proposal*, and the proposal itself still has to have passed independent authorization before it was even shown — Agentforce v2's Executor still re-checks permissions even after human approval, it doesn't treat approval as a substitute.)

2.5.9 The grounding threat model — eight named attack vectors, worth having cold

Retrieval isn't just a quality problem — it's its own trust boundary with its own attack surface, distinct from prompt injection generally (2.5.4) and from tool-level attacks (2.6). Eight vectors worth being able to name and mitigate on

demand:

1. **Retrieval poisoning** — an attacker seeds a document crafted to rank highly for specific target queries, so it gets pulled into context even though it's not actually authoritative. Mitigation: restrict *who* can create indexable content in the first place — a content-governance control, not a retrieval-ranking fix.
2. **Prompt injection via retrieved data** — the indirect-injection case from 2.5.4, specifically through the grounding path. Mitigation: prompt-defense policies applied to retrieved content as a class, not just to user input.
3. **Stale grounding** — the index hasn't been refreshed since the source changed. Mitigation: read-after-write for anything grounding-sensitive (this is 4.1's consistency discussion, restated as a security-relevant staleness risk, not just a correctness one).
4. **Cross-user leakage** — retrieval isn't scoped to the requesting user at all.
5. **Cross-tenant leakage** — the index or vector store is shared across orgs without real isolation. Mitigation: tenant isolation, ideally with customer-managed keys — the same orgJWT-per-tenant principle as 5.1, restated for the index itself rather than just the query path.
6. **Unauthorized retrieval** — a customer's own retriever implementation bypasses sharing rules. Mitigation: enforce the *running user's* access (FLS, CRUD) inside the retrieval call itself, never assume an upstream check already covered it.
7. **Metadata leaks** — a chunk includes prepended/internal fields that were never meant to reach the model or the user.
8. **Malicious documents** — an uploaded file carries hidden text, homoglyphs, or embedded instructions designed to be extracted during parsing. Mitigation: parser hardening — treat document parsing itself as a place attacker-controlled content can smuggle a payload through, not just the extracted text.

The line that ties all eight together, worth stating verbatim: "*Every grounding result has to pass through the platform's own sharing, field-level-security, and CRUD model before it reaches the prompt — retrieval doesn't get a side door around the permission system just because it's a vector search instead of a SOQL query.*"

Two distinctions worth having crisp, because they're easy to blur under pressure:

- **Masking and authorization solve different problems.** Authorization controls *which data you may access at all*; masking controls *which sensitive fields are exposed even after legitimate access*. A field can be authorized and still masked (you can see the record, not the SSN on it); the two checks are independent and both have to pass.
- **Grounding improves knowledge quality; it does not improve — or substitute for — authorization** (this is 2.14's lesson 7, and it's exactly the mistake retrieval vector #6 above describes: assuming that because retrieval *found* something relevant, the caller was *entitled* to it).

2.6 The agent/tool interaction model — MCP trust specifics

- **The model proposes; deterministic systems dispose.** A model "calling a tool" is a structured request — a name and arguments — never an executed effect. The runtime decides whether and how to execute it.
- **Tool descriptions are untrusted supply-chain input.** The text a tool exposes to the model to describe when/how to call it is itself read and acted on by the model — a compromised or malicious description is a viable attack vector ("**tool poisoning**"). Descriptions should be pinned, hashed, reviewed like any dependency, with drift after approval treated as suspicious ("**rug pull**").
- **Tool outputs are untrusted input on the way back in.** Anything a tool returns re-enters the model's context and can carry an injected instruction, exactly like a malicious retrieved document — validate/sanitize before trusting for further reasoning or showing the user.
- **A standardized tool-calling protocol (MCP)** exposes three primitive surfaces: callable **tools**, readable **resources**, reusable **prompt templates** — over a base RPC protocol, with a capability-negotiation handshake, and an authorization model built on **delegated, audience-bound tokens** (a token minted for one server cannot be replayed against another) plus an explicit prohibition on blindly forwarding a client's token upstream.

- **Connecting to any external tool server extends your trust boundary to that server's code and everything upstream of it.** Trust is not transitive, not static — a server can change a tool's behavior after review. Controls: an explicit allowlist of connectable servers, least-scoped and audience-bound credentials per connection, treating both descriptions and outputs as untrusted, periodic re-review for definition drift.
- **Fine-grained vs. broad/generic tools is the single most consequential interface design decision.** A generic "update any field on any record" tool hands the model a privilege-escalation primitive: the model effectively parameterizes what data gets touched. The mature middle ground when a generic tool is unavoidable: a **capability-scoped generic tool** — constrained by an explicit allowlist of the specific objects/fields it may touch in context, hard value constraints enforced as policy (not prompt instruction), platform authorization still the non-negotiable backstop underneath the model's choice. (This is precisely Generic CRUD Actions, 1.5/2.5.6's parenthetical — have that mapping ready cold.)
- **Multi-agent (supervisor/specialist) architectures** trade a larger coordination and trust surface (every inter-agent message is a new untrusted channel; a specialist's hallucination can propagate unchecked if the supervisor over-trusts it) for separation of concerns and parallelism. Prefer a single, well-scoped agent unless the domains are genuinely distinct; if multi-agent is used, treat every inter-agent handoff exactly like an external, untrusted input. (Agentforce v2's own justification for Supervisor/Specialists, 1.3, is exactly "separable cognitive loads," not agents-for-the-sake-of-it — the same bar this principle sets.)

2.6.1 MCP's own architecture — three parts, worth being able to draw

- **Host** — acts as the container and coordinator. Creates and manages multiple client instances, enforces security policy and consent requirements, and holds the **full conversation history** — the host is where cross-server context actually lives, not any individual client or server.
- **Client** — each client maintains an isolated connection to exactly one server and one stateful session per server; the client is where protocol negotiation and capability exchange happen.
- **Server** — provides specialized context and capabilities by exposing three primitives: **resources** (readable, declared by the server — pieces of context it makes available), **tools** (actions/functions the model can invoke), and **prompts** (reusable templates the server declares; the client can list them).

Transport: JSON-RPC 2.0 — every message carries `id`, `method`, `params` (plus the `jsonrpc: "2.0"` version tag).

Initialization lifecycle — the actual handshake, in order:

1. Client sends `initialize` with its `protocolVersion`, `capabilities`, and `clientInfo`.
2. Server responds with its own protocol version, declares its capabilities, and identifies itself.
3. Client confirms with a notification.

Two things worth stating explicitly if he asks "why does this matter": version negotiation happens once but its result is effectively carried on every subsequent request; and **capability negotiation is the actual heart of initialization** — it's what determines which of the primitives above (resources/tools/prompts, plus client-side roots/sampling/elicitation) either side is even allowed to invoke for the rest of the session, not a formality before the "real" protocol starts.

Client-side primitives, the less commonly discussed half of MCP: roots, sampling, and elicitation — the server can use **sampling/createMessage** to ask the *client's* own model for help generating something, and **elicitation** to ask the *human* for more information mid-tool-call. Concretely: the server "asks the host's brain for help" rather than assuming it has to solve everything itself — this is a real inversion worth naming if he wants to know whether you understand MCP is bidirectional, not just "client calls server's tools."

Two more server-declared capabilities worth knowing by name: logging (tracked by the server, sent to the client in a structured form) and **completion** (argument autocomplete, offered by the server for its own tool/prompt arguments).

2.6.2 Attacks at the MCP-server level — three categories, each with its own fix

This is the layer beneath the general "connecting to an external tool server extends your trust boundary" principle above — the concrete taxonomy of *how* that extension goes wrong:

1. **Malicious server** — e.g. a "PDF summarizer" tool that quietly posts every document it touches to an attacker-controlled endpoint. Mitigations: anomaly detection on argument volume (a tool suddenly handling far more data than its stated purpose implies is a signal); a deterministic, idempotent scanning pipeline around anything a tool ingests; least privilege and least sandboxing scope for what the tool can reach; and **no token passthrough** — never forward a caller's own credential to a downstream server unmodified (2.5.3's mitigation #4, restated for MCP specifically).
2. **Compromised server** — a previously-trustworthy server ships an update that introduces malicious code. Mitigations: pin the server's version and dependencies rather than always-latest; scan the server and its SBOM (software bill of materials) before trusting an update; least privilege and sandboxing as the standing defense even if a compromise slips through.
3. **Impersonation of a server** — a connection claims to be a trusted server but isn't. Mitigations: verify the connection; **bind tokens** to the specific server/audience they were minted for (the audience-bound-token principle above, worth reading up on the specifics of RFC 8705/9068-style token binding if he goes deep here); pin and verify server identity rather than trusting whatever answers at a given address.

2.7 State and persistence

Kind of state	Scope	Trust level	Key risk
Working memory	Single reasoning turn	Ephemeral, in-context	Context-window overflow, truncation
Conversation memory	Single session	Session-scoped	Cross-session leakage if not properly isolated
Checkpoint/proposal state	A specific pending action	Durable, transactional	Must survive retries and partial failure
Episodic memory	Past interactions, summarized	Durable, longer-lived	Can encode a wrong "remembered" fact that drives future bad actions
Semantic/long-term memory	Facts about an entity across sessions	Durable, cross-session	**The most dangerous class** — a poisoned write here replays an attack on every future session

Principles: long-term memory inherits full data-governance obligations (access control, retention limits, deletion/right-to-erasure) because it's durable stored data, not a debugging convenience. **Memory is a write-policy-gated surface, not an automatic side effect** — what gets promoted from a session into durable memory should be validated/classified before being stored as a trusted fact, or an indirect-injection payload written once replays indefinitely. **Durable truth lives in the system of record, not in the model or its memory** — anything remembered should be a hint to re-verify against live data before it drives a consequential action, never itself authoritative. (Tie-in: this is exactly why Agentforce v2's `TenantGrounding` and live data queries hit the **live org directly** during planning rather than relying on the pre-ingested vector index for anything state-sensitive — Section 4.1 below.)

Three protection rules worth stating as a checklist, not just a principle: user-and-tenant isolation on every memory read/write; every memory must carry a **defined lifetime and expiry** — nothing durable by default; and every stored memory should **cite where it came from**, so a downstream consumer can trace a "remembered" fact back to its source rather than trusting it as free-floating ground truth.

Five authority levels worth distinguishing when reasoning about *whose* authority a given action actually runs under: End User → Service → Delegated → System → Mixed. Most of Section 2.5's guidance is really the argument for staying as far left on this spectrum as the operation allows — "End User" is the strongest guarantee

(2.5.3's strongest confused-deputy mitigation), and each step right is a deliberate, narrowly scoped exception, not a default.

Five non-negotiables specifically for multi-agent (supervisor/specialist) architectures — extends 2.6's "treat every inter-agent handoff as untrusted" principle into a concrete checklist:

1. Route on semantics, validate on output — a supervisor's routing decision is a guess about intent; only the specialist's actual output gets trusted, and only after validation.
2. Give each specialist the least context it needs to do its one job — not the full conversation by default.
3. Least privilege and authority, scoped down per hop — a specialist's authority should never be broader than what its one job requires, even if the supervisor itself holds more.
4. Tag every inner sub-agent's claim back to the supervisor — attribution has to survive the hand-off, not just exist at the top level.
5. Every side effect must go through the same idempotency checkpoint, regardless of which specialist in the graph triggered it — idempotency is a property of the *system*, not of any one agent's code path.

2.8 Concurrency and consistency

- **No global transaction spans the model, the tool layer, and downstream systems.** Design explicitly for partial success — a multi-step action can succeed on step one and fail on step two, with no built-in rollback.
- **Time-of-check to time-of-use (TOCTOU) is a first-class risk.** State read during planning ("does this record still look eligible?") can change before the write executes. Authorization and business invariants must be re-evaluated **at the point of mutation**, atomically, never trusted from an earlier read. (This is the general form of ResolveFlow's `execute()` posting `state["proposed_action"]` **verbatim** rather than recomputing it after approval — the general TOCTOU fix and the specific implementation are the same idea.)
- **Optimistic concurrency** (a version token captured at read time, checked and updated atomically at write time) is the default pattern against lost updates when two operators or two concurrent agent turns touch the same entity. For anything more complex than a single record (a computed "effective permissions" view derived from many contributing records), a **composite version token** — a hash over every contributing record's version — is needed so a write is rejected if *any* input to the original decision has since changed.
- **Pessimistic concurrency** is the other tool in the kit, not a worse version of optimistic — reach for it specifically when the work between read and write includes an **external callout** you can't cheaply retry (an outbound API call mid-transaction, for instance): acquire an exclusive lock before touching the record, hold it until commit, block any other writer for that window. Optimistic concurrency assumes conflicts are rare and retry is cheap; pessimistic concurrency is the right choice exactly when retry *isn't* cheap because a real side effect already happened in between.
- **Retries require idempotency.** Network hops and model calls fail and get retried; without an idempotency key derived from the logical operation, a retry can double-apply an effect. Standard pattern: claim an idempotency key atomically before doing the work; if already claimed, return the previously recorded result instead of re-executing. **The key itself needs to be bound to more than just the operation** — a key derived only from the logical operation (e.g. `createUser`) is reusable by a *different* caller unless it also incorporates the invoker's own identity: something like `SHA(operation + invoker_credentials)`, so the key can't be replayed or reused across principals. For concurrent requests specifically, don't compute-then-check for a duplicate — **insert the key under a unique database constraint** and let the constraint itself reject the second concurrent attempt; a check-then-insert has its own race window, an insert with a unique constraint doesn't.
- **Idempotency has a layer problem, not just a mechanism problem.** Request-level idempotency (dedup on a key per HTTP call) doesn't automatically give you business-level idempotency — once a step touches multiple downstream systems, DB-level dedup on your own table isn't enough on its own, because a crash between "your DB committed" and "the other system was told" leaves the two systems disagreeing with no local record of the gap.
- **Serialize writes per logical entity, parallelize everything else.** Reads and writes to independent entities run concurrently; writes targeting the same entity must be ordered (a per-entity queue or lock). (Directly

informs the Parallel Action Execution proposal — Section 3.4 below — which only parallelizes independent reads; it explicitly does not attempt this for independent writes yet.)

- **Sagas / compensating actions** replace atomic multi-step transactions across systems that don't share a transaction manager: each step should be independently reversible or reconcilable, not assumed to succeed together.
- **The transactional outbox pattern is the concrete fix for the dual-write problem above** — worth being able to draw from memory: one database transaction both makes the business change *and* inserts a row into an outbox table (`BEGIN → create/update record → insert outbox event → COMMIT`, atomically, as one transaction); a separate, independent relay process then reads the outbox and publishes the event to the other system, marking the row delivered only after a confirmed publish. This turns "update my data and tell another system it happened" from two operations that can fail independently into one atomic local commit plus an at-least-once, retryable relay — and because the relay redelivers on any doubt, **the thing it delivers to had better be idempotent to receive**, which is exactly why compensating actions themselves need to be idempotent too: if a compensating "revoke this grant" call times out without you knowing whether it landed, retrying it should be safe by construction, not something you have to reason about case by case.
- **Verify the post-condition; don't assume the effect.** After a mutation — especially one with multiple possible paths to the same end state — re-derive the intended end state and assert it actually holds. A 200-OK is not proof the world now looks the way you intended. (Pipeline stage 7, 1.2.)

Six rules for reasoning about concurrency under pressure, worth having as a single recitable list:

1. Parallelize independent reads.
2. Serialize conflicting writes.
3. Every write is compare-and-swap, not blind overwrite.
4. Never hold a lock, or sit inside a transaction, across an LLM call or any other slow external callout — that's an availability incident waiting to happen, not a correctness fix.
5. Read your own writes — don't let a client that just wrote see stale state from a replica or cache on its very next read.
6. Guard invariants explicitly: route consistency-sensitive reads to the primary, not a replica; return the actual post-write state from the tool call itself rather than re-deriving it; and version-fence reads so a stale read can't silently drive a write.

2.9 Failure modes by layer

Layer	Representative failure	Typical symptom	Primary mitigation
Channel/experience	Duplicate submission on perceived hang	Duplicate side effects	Idempotency keys per turn
Agent configuration	Overlapping/ambiguous task-scope definitions	Non-deterministic routing	Disjoint scopes, precise routing descriptions, eval gating
Runtime/orchestration	Planning loop (oscillating re-plans)	Latency spikes, cost blowup	Step budgets, loop detection, deterministic fallback
Reasoning (model)	Hallucinated content or a fabricated tool argument	Confident, wrong, well-formed answer, no error	Grounding + citation enforcement; strict schema validation at the gateway
Tool/action execution	Unvalidated argument (an absurd limit value)	Resource exhaustion or unsafe write	Range/enum validation at the gateway
Data/grounding	Stale retrieval index	Agent cites outdated info with a "passing" grounding check	Freshness monitoring, ingestion-lag alerts
Multi-agent coordination	Cascading retries across specialists	Fan-out amplification	Circuit breakers, bulkheads per specialist
Infrastructure	Model provider degradation	Broad latency/error spike	Fallback routing, graceful degradation

A complementary taxonomy, specific to the intent/topic/action-selection layer — worth having as a fast enumerable list if he asks "what actually goes wrong in practice, not in theory": misclassification; no topic match; multiple competing topic matches; a hallucinated or missing required parameter; a schema or enum violation; an ambiguous entity reference (which "John Smith"?); a governor/rate-limit hit mid-plan; a grounding check that silently fails open; and prompt injection delivered specifically via grounded content rather than user input. Nine failure shapes, each wants its own detection signal rather than one generic "error rate" metric — most of them produce a confident, well-formed, wrong answer, not an exception.

A cross-cutting, often-missed category: slow-burn/silent degradation — failures that never trip a single threshold alert but compound over days or weeks: gradual quality decay from small prompt/index/model changes stacking, refusal-rate creep, config drift between environments, memory/eval-set staleness. Requires monitoring **rates of change against a trailing baseline**, not just absolute thresholds — each signal can look "green" in isolation while the aggregate trend is a real regression.

The single most important failure-analysis habit, memorize verbatim: "*A well-formed, confident, HTTP-200 answer can be entirely wrong.*" Conventional error-rate monitoring is blind to this class of failure — unique to systems where the decision-maker is a language model, not deterministic code.

2.10 Observability

Conventional tracing (traces/logs/metrics) still applies, plus arguably a fourth pillar: **replay artifacts** — a captured record of exactly what the model saw (rendered prompt, retrieved context, tool schemas) for a given decision, because reconstructing "why did it do that" is a recurring operational need a plain trace doesn't satisfy.

Key differences from a conventional service trace:

- **The trace is session-rooted, not request-rooted** — spans a whole multi-turn conversation, turns and reasoning/tool spans nested beneath.
- **The shape of the trace is itself a signal.** Because control flow is synthesized at runtime, a shift in the *distribution* of paths taken (more re-plans, a different tool mix, more refusals) is often the earliest regression indicator — frequently before any error-rate metric moves. (Story C above is exactly this kind of investigation, applied.)
- **High-cardinality fields (user IDs, free-text arguments) must never become metric labels** — they belong in traces/logs, sampled and indexed, or you'll overwhelm a metrics backend.

A minimal useful signal set: refusal rate (with confidence context), **tool-selection entropy** (a distributional drift detector — unexpectedly flat or shifted tool usage often means a prompt/catalogue regression), grounding-check failure rate, re-plan/loop count, permission-denied rate (a spike can be either an outage *or* an attack probe), anomalous argument values.

Alerting discipline: separate *page-worthy* (user-visible harm or security signals — SLO burn rate, guardrail-bypass, permission-denied spikes) from *dashboard-only* (distributional/quality trends). Correlate every alert/dashboard with a change marker for each of **five independently-versioned axes** — model version, prompt version, tool-catalogue version, index version, configuration — since the most common root cause of a regression is a change on one of these axes, and without a change annotation, diagnosis time balloons. (This is the generalized version of Story B's "each of the five workflows is an independent rollout unit" and Story C's root-cause isolation method.)

Traces are sensitive data, not ordinary application logs — they routinely contain user utterances, retrieved record content, tool arguments, potentially regulated data. Mask at capture time (before writing to the trace store), tier retention (aggregated metrics long-lived; masked traces medium-lived; unmasked debugging tier short-lived and tightly access-controlled; security audit logs long-lived and immutable), and ensure the trace store can honor a data-deletion request, since it holds personal data like any other store.

2.11 Scalability

An enterprise agent is a **distributed fan-out topology**, and its overall latency is best understood as **multiplicative in the number of reasoning turns, not additive in pipeline stages**. A request requiring the planner to select a tool, observe the result, and select a second tool pays for two full model round-trips plus two guardrail passes — budget latency per-turn and sum over the expected turn count for a given workflow.

Key latency components:

- **Inference latency** decomposes into time-to-first-token (dominated by prompt/prefill length) and inter-token latency (dominated by output length). The practical consequence: **the length of the system prompt and grounded context is a tax paid on every single turn** — prompt minimization, output-length capping, and prompt-prefix caching are the primary levers.
- **Retrieval latency** depends on index size and query selectivity; a **semantic cache is dangerous in a multi-tenant, permission-scoped system** unless the requesting principal's authorization context is folded into the cache key — two users can phrase a near-identical question while having very different rights to the answer, and a naive query-text-only cache can leak one user's data to another. (Story C's stale-grounding cache bug is this exact mistake, applied to staleness instead of tenant leakage — same root cause, watch for both.)
- **Action/tool latency** depends on the downstream system called; a slow synchronous tool call can starve the platform's overall concurrency budget if long-running requests aren't isolated or capped.
- **Backpressure is a security control as much as a performance one.** Without admission control, a burst of agent-driven requests can exhaust downstream concurrency and degrade the whole platform — a self-inflicted denial-of-service condition. (Directly informs Section 4.3's governor-limits discussion.)

Illustrative latency budget for a representative multi-turn workflow (illustrative, not measured figures from any real system — useful for reasoning about *shape*, not for quoting as fact):

Stage	p50	p95	p99
Ingress + auth	~20 ms	~60 ms	~120 ms
Input guardrail scan	~40 ms	~120 ms	~300 ms
Planning inference (turn 1)	~700 ms	~1.8 s	~3.5 s
Retrieval	~120 ms	~400 ms	~900 ms
Planning inference (turn 2)	~650 ms	~1.7 s	~3.3 s
Authorization check	~60 ms	~180 ms	~400 ms
Action execution	~150 ms	~500 ms	~1.2 s
Output guardrail scan	~50 ms	~150 ms	~350 ms
Response synthesis (turn 3)	~500 ms	~1.3 s	~2.6 s
End-to-end (excl. human approval wait)	**~2.3 s**	**~6.4 s**	**~13.3 s**

The reading that generalizes: reasoning turns dominate both the median and the tail; collapsing multi-turn plans into fewer round-trips is the single biggest latency lever, traded against harder guardrailing per turn. The p50-to-p99 spread is characteristically wide for LLM-backed pipelines — **quote SLOs at p95, set client timeouts comfortably above p99**, never around the average. (This is the general version of Agentforce v2's own disclosed "Thinking…" latency problem, Section 3.3 — say so if he connects the two.)

2.12 Design trade-offs

Trade-off	Choose the first when...	Choose the second when...
Retrieval (RAG) vs. direct structured lookup	Answering fuzzy, natural-language questions over unstructured docs	The fact must be exactly correct and current ("what access does this person have right now") — never let retrieval output stand in for an authorization decision

Trade-off	Choose the first when...	Choose the second when...
LLM-planned agent vs. deterministic workflow	Open-ended intent capture, flexible front door	Everything after intent is captured — the execution plan should be deterministic, testable code
Standardized tool-protocol server vs. plain internal API	The tool surface is consumed by models across multiple hosts	The consumer is conventional software with no need for model-facing discovery
Generic tool vs. bespoke narrow action	Low-consequence, high-variety reads or light, tightly-constrained writes	Any high-consequence, irreversible, or regulated operation
Model-evaluated policy vs. deterministic policy engine	Never, for authorization decisions	Always, for authorization — a model may *suggest*, permit/deny must come from code
Caller's own credentials vs. a service/integration identity	The caller has a downstream identity — the default	A narrow, unavoidable elevation is required, kept as tightly scoped as possible
Single agent vs. multi-agent	The domain is genuinely one cohesive scope	Distinct specialized domains justify the added coordination/trust-surface cost
Long context ("stuff it all in") vs. retrieval-on-demand	Rarely — small, non-sensitive context	Anything large or sensitive — a large in-context payload both costs more and expands the injection/exfiltration surface
Synchronous vs. asynchronous execution	Reads and auto-approved low-risk writes	Anything requiring human approval or bulk processing

The unifying principle behind nearly every row: push flexibility toward the reasoning layer, push determinism toward the execution layer. The model should have latitude to interpret intent; it should have almost none deciding-and-acting on consequential operations.

2.13 Pseudocode you should be able to write live

If he hands you a laptop, these are far more relevant than a random LeetCode problem — type each once before Friday.

Deterministic authorization middleware, independent of the model's output:

```
def intercept_tool_call(call, caller):
    if not caller.is_authenticated():
        return deny("unauthenticated")

    required_capability = map_tool_to_capability(call.tool_name, call.args)
    if not authz_service.caller_holds(caller, required_capability):
        audit.record(caller, call, "DENY_NO_CALLER_AUTHORITY")
        return deny(f"caller lacks: {required_capability}")

    decision = policy_engine.evaluate(caller, call, required_capability)
    audit.record(caller, call, decision)  # log the decision before any effect
    if decision.effect == "PERMIT":
        return proceed(decision.obligations)
    elif decision.effect == "REQUIRE_APPROVAL":
        return route_to_approval(call, decision)
    else:
        return deny(decision.reason)
```

Idempotent execution wrapper:

```
def execute_once(proposal, do_work):
    first_time = idempotency_store.try_claim(proposal.key, proposal.id)
    if not first_time:
        return idempotency_store.await_result(proposal.key, timeout=30)
    try:
        result = do_work()
        idempotency_store.record_result(proposal.key, result)
        return result
    except Exception as e:
        idempotency_store.record_failure(proposal.key, e)
        raise
```

Optimistic concurrency guard:

```
def mutate_with_version_check(entity_id, expected_version, mutation_fn):
    current = data_store.read(entity_id)
    if current.version != expected_version:
        raise ConcurrencyConflict("state changed since read; re-inspect before mutating")
    return mutation_fn(entity_id)  # conditional on the version check above
```

Permission-aware retrieval — pre-filter, not post-filter:

```
def acl_aware_search(query, caller, k=8):
    query_vector = embed(query)
    visible_scopes = caller.authorization_labels()
    hits = vector_index.search(
        vector=query_vector,
        top_k=k,
        filter={"scope": {"in": visible_scopes}},  # enforced INSIDE the search
    )
    # belt-and-suspenders: re-check live authorization on the result set
    return [h for h in hits if caller.can_read(h.resource_id)]
```

Tool-selection entropy as a drift detector:

```
import math

def tool_selection_entropy(tool_counts: dict) -> float:
    total = sum(tool_counts.values())
    if total == 0:
        return 0.0
    entropy = 0.0
    for count in tool_counts.values():
        if count == 0:
            continue
        p = count / total
        entropy -= p * math.log2(p)
    return entropy  # compare against a trailing baseline; alert on large deviation
```

Deterministic policy decision (illustrative — production systems typically use a dedicated policy-as-data engine):

```
def evaluate(request) -> Decision:
    if request.action == "grant" and request.target_capability in SENSITIVE_SET:
        if not request.caller_holds_admin_capability:
            return Decision("DENY", "sensitive grant requires elevated authority")
        return Decision("REQUIRE_APPROVAL", "sensitive grant")
    if request.batch_size > MAX_BATCH:
        return Decision("DENY", "exceeds blast-radius limit")
    if request.action == "inspect":
        return Decision("PERMIT")  # reads are already access-constrained upstream
    return Decision("REQUIRE_APPROVAL", "default-deny-to-human for writes")
```

2.14 Twenty-two principal-level lessons — recite these when asked to summarize your philosophy

1. The model proposes; deterministic systems authorize.
2. Retrieved content is data, never authority.
3. Authentication and authorization are different questions.
4. Authorization belongs at execution time, against live state — not at plan time.
5. Never trust the planner as a security boundary.
6. Tool descriptions and tool outputs are both untrusted input.
7. Grounding improves factuality, not authorization.
8. Every write must be idempotent or safely retryable.
9. Human approval complements authorization; it never replaces it.
10. Use the caller's own authority wherever possible.
11. Autonomy should decrease as consequence increases.
12. Never grant a capability you cannot independently constrain.
13. Deny by default; permit by exception, for every action and every actor.
14. Minimize what enters the model's context window.
15. Prefer narrow, verb-specific tools over generic operations.
16. Log the decision, not just the outcome.
17. Bound the blast radius of every action.
18. Attribute every action to a real, accountable identity.
19. Guard shared state with explicit concurrency control, never "the model will notice."
20. Fail closed, degrade gracefully.
21. Separate the reasoning plane from the control plane as independently deployable units, not just a logical distinction inside one process.
22. Design for the adversarial case first, then relax — never the reverse.

2.15 The tiered question bank — self-test in order of difficulty

Foundational: What's the difference between the model's role and the runtime's role when a "tool call" happens? Why isn't a well-formed, confident, error-free answer sufficient evidence an agent behaved correctly? Soft vs. hard guardrail — which one carries security guarantees?

Senior: A retrieval index has gone stale — concrete failure mode and correct mitigation? Why must side-effecting tools be idempotent, and how do you implement that in practice? Walk through what "the model decides to call a tool" actually triggers, step by step, up to a real side effect. Direct vs. indirect prompt injection, with a concrete example of each.

Staff: Intent classification is probabilistic and will sometimes misroute — your containment strategy, independent of routing accuracy? "We validate permissions in the tool" — where exactly, is it deterministic, what happens if the

check errors, how does it behave under concurrent access? Walk through correct handling when a tool call to an external system times out but may have actually succeeded. Name the trust boundaries in a typical deployment and what each requires for authn/authz.

Principal: Design the trust and consistency model for adding long-term, cross-session memory — what breaks at scale? Two concurrent operations touch the same entity — what breaks, and how do you make the system correct without relying on the model to coordinate? Design the observability needed to reconstruct a single production turn end-to-end, reconciled with data-minimization constraints. Argue against "just give the agent broad access so it can help with anything," from a blast-radius and confused-deputy standpoint. Compare bespoke narrow actions against generic broadly-parameterized tools — when is each appropriate, and what changes about the security analysis?

Security-architect: Connecting to an external tool server over MCP — interrogate the trust model: server trust, description trust, token scope, output handling, change control. Walk through a complete confused-deputy attack end-to-end and name every independent control that should stop it, so no single failure is catastrophic. What does data masking protect against, what are its limits, how does it interact with an active exfiltration attempt? Define "fail closed" precisely across authorization, guardrails, dependency failures — give a concrete example where "fail open" would be catastrophic. You're signing off on an autonomous, revenue-affecting agent — enumerate the non-negotiable controls and the residual risks you'd still document even after they're all implemented.

2.16 The closing litmus test — use this to defend any design choice

> **"If the model is fully compromised by an attacker right now, what is the worst thing it can actually cause to happen?"**

If the answer is bounded, small, attributable, and reversible, the architecture is sound. If it isn't, some layer is trusting the model as a security boundary. **Run every claim you make in this interview through this question before you say it out loud** — it's the fastest way to catch yourself overclaiming, and it's exactly the kind of question a fiercely technical, security-company interviewer is listening for you to already be asking yourself.

---

3. Scalability — the questions a fiercely technical interviewer asks after the

architecture tour

Don't wait for "how does this scale" as one question — have each of these as a separate, specific answer, because a real interviewer will pick apart each subsystem's scaling story independently. (Section 2.11 above has the vendor-neutral latency/scaling framework — this section is Agentforce v2's specific instance of it.)

3.1 The action catalog — covered above (1.3, Story A). One-line recap if he circles

back: "*Funnel decouples LLM cost from catalog size; only vector-index size and search latency grow with it, and ANN search is built to stay fast at scale.*"

3.2 The knowledge base and retrieval tier — the HLD's own disclosed risk

State this proactively if scalability comes up — it's a real, disclosed limitation: "*Knowledge retrieval from a centralized C360 Data Cloud instance is still an open scalability question in the current design — it hasn't been proven out at full multi-tenant scale yet.*" Then reason about *why* it's a legitimate concern and what you'd probe if asked to own that risk:

- **Two separate indexes with different scaling profiles.** The Unified Knowledge Dataspace (product docs) is shared across all customers and managed by content teams — its scaling problem is query throughput

against a large-but-slowly-growing shared corpus. The NLD/API-grounding index is different: it's vectorized *per-org* metadata layered on top of a shared NLD corpus, so its scaling problem is closer to multi-tenant index growth — as org count grows, does per-tenant metadata get its own logical partition, or does everything sit in one shared index disambiguated only by a tenant filter at query time? (This is exactly the ACL-before-retrieval question from Section 2.5.7/5.2, but it's also a **scaling** question: a shared index with a tenant filter applied at query time still has to search *the whole index* before filtering, which gets slower as total tenant count grows, even though any single query only cares about one tenant's slice.)

- **The honest answer if pushed on "so what would you do":** "*I'd want to know whether the NLD/metadata index is a single shared index with tenant-scoped filtering or logically partitioned per tenant/shard before proposing a fix — those have different failure modes at scale. If it's shared-with-filter, the natural fix is tenant-aware sharding (Section 4.5 below) so a query only ever touches its own tenant's partition. If it's already partitioned, the bottleneck is more likely ingestion throughput (the mDLO extraction pipeline, Section 4.1) than query-time search.*" Don't overclaim a fix you haven't verified is the actual bottleneck — volunteering the right *diagnostic question* is more credible than a confident wrong answer.

3.3 Planning latency — the "Thinking..." screen problem

The concrete problem, verbatim from the design's own framing: `compilePlan` is I/O-bound (it's making retrieval calls and an LLM call) and can take real time — risking a static "Thinking…" screen that reads as broken, not just slow. This is Agentforce v2's specific instance of Section 2.11's general "reasoning turns dominate the tail" latency problem.

The architectural answer — "Thinking Out Loud": stream intermediate results (the high-level plan as it forms) rather than blocking on the full compile. This is explicitly why the Reasoner is internally designed as a **granular, multi-phase** process (Requirement Synthesis → Scaffold Ingestion → Plan Compilation, each its own sub-stage) instead of one monolithic call — the architecture is "streaming-ready" by construction, even before a first-class streaming capability is fully wired through the UI layer. **The general principle to state:** "*Perceived latency and actual latency are different problems with different fixes. If you can't make the compile call faster, you can still make the wait feel shorter and more trustworthy by surfacing real intermediate state as it's produced — which only works if your architecture already computes that intermediate state as discrete, inspectable stages rather than one opaque call. Designing for streaming later means designing granular stages now, even before you've built the streaming UI.*"

3.4 The serial-execution bottleneck — and the fix you should be able to design live

The concrete problem (a real example, use it exactly): a diagnostic plan like "show me all permission sets, public groups, and queues assigned to User X" naturally decomposes into one setup step (find the user) followed by **three independent** data-gathering steps — but under the current Executor model, those three steps run **serially**, purely because nothing in the plan schema declares them as parallelizable.

The proposed fix — you should be able to draw this schema from memory:

```
{
  "executionPlan": {
    "stages": [
      {
        "stageId": "stage_1_find_user",
        "execution_mode": "serial",
        "steps": [ { "stepId": "step_1_find_user" } ]
      },
      {
        "stageId": "stage_2_gather_assignments",
        "execution_mode": "parallel",
        "steps": [
```

```
          { "stepId": "step_2_find_perm_sets" },
          { "stepId": "step_3_find_public_groups" },
          { "stepId": "step_4_find_queues" }
        ]
      }
    ]
  }
}
```

How the mechanism has to work, if he asks you to design the Executor-side runtime for this (this is a genuine live-design prompt he could hand you — and it's exactly Section 2.8's "serialize writes per logical entity, parallelize everything else" principle in miniature):

- **The Reasoner must do dependency analysis at compile time**, not the Executor at run time — grouping steps into stages based on whether a step's inputs depend on another step's *output* within the same plan. Two steps are parallelizable only if neither consumes the other's result and neither writes to a resource the other reads/writes (a straightforward data-flow dependency graph — this is the same problem as instruction scheduling or a build-system's task graph).
- **The Executor runtime, on hitting `execution_mode: "parallel"`, fans out all steps in that stage concurrently and joins before advancing** — the join is a hard synchronization point; the next stage cannot start until every step in the current one completes (or definitively fails).
- **Partial failure inside a parallel stage is the actual hard part, and it's the question he's most likely to push on:** if `step_3_find_public_groups` fails while `step_2` and `step_4` succeed, what happens? Since these are independent *reads* in this example, the safe default is: let the successful reads complete, surface the one failure explicitly rather than silently dropping it, and let the response layer show partial results with the gap clearly marked — this only works because read failures are safely partial-tolerant. **If the parallel stage contained independent *writes* instead of reads, this gets much harder** — you'd need either per-step idempotent retry with no cross-step ordering requirement, or a decision about whether the whole stage is all-or-nothing (which then needs a compensating-action/rollback story for the steps that already succeeded — a **saga pattern**, Section 2.8) — and the honest answer, if pushed here, is that read-only parallel stages are the safe first version of this feature, and parallel writes are a substantially harder problem the design doesn't claim to have solved yet.

3.5 Cost and the long tail

The business framing is "30–40% of use cases represent over 90% of user need, but are too numerous to hard-code" — that number has a *cost* consequence worth being able to state: hard-coding a bespoke Java action per capability doesn't scale economically past a fairly small number of high-frequency "hero" cases; the entire Knowledge-as-Code bet is that the marginal cost of adding the *N+1*th capability should be "author one Markdown file, get it human-reviewed" rather than "ship new code" — a fundamentally different cost curve as the long tail grows. **The tie to your own eval work:** every new NLD is a candidate to silently regress something else in the shared action catalog (a new NLD's candidate actions could crowd out an existing NLD's in the funnel's shortlist) — which is exactly why the eval harness has to run **as regression on every content change**, not just every code change; content is now load-bearing in a way it wasn't when capabilities were bespoke Java. (This is a direct instance of Section 2.5.6: "treat every tool-schema/content change as a security-boundary change, not just a functional one.")

---

4. Distributed-systems mechanics, applied to this exact system

`../CrowdStrike_Prep_Deep_Dive.md` §3 has the from-scratch mechanism (CAP/PACELC derivations, rate-limiting algorithms, consistent hashing, delivery semantics) if he wants you to derive something live. Section 2.7/2.8 above has the vendor-neutral applied framework (state trust tiers, TOCTOU, optimistic concurrency, idempotency, sagas).

This section is the **doubly-applied** version, tied specifically to Agentforce v2's own real components — this is what separates "I know the CAP theorem" from "I can reason about where it bites in a system I actually own."

4.1 Consistency: the mDLO ingestion pipeline is an eventually-consistent system, and you should say so unprompted

The real ingestion state machine, verbatim: `New → Processing → Syncing → Data_Copied → Ingestion_Requested → Ingestion_Complete`. An admin edits an NLD (or, more importantly, the org's *own* metadata changes — a new field, a changed sharing rule) and that change has to propagate through this pipeline into the vector index before the agent's grounding reflects it.

The question to expect: "*What happens if a user asks the agent to act on something they just changed, while that change is still mid-pipeline?*"

The honest, precise answer: "*This is a classic staleness window in an eventually-consistent system — the live org (source of truth) and the grounding index (the agent's view of it) are not the same store, and the pipeline between them is asynchronous by design, not a bug. Two different failure shapes to distinguish: (1) conceptual/NLD knowledge staleness — an admin updates a Workflow NLD and the agent briefly uses the old version; low-stakes, because NLDs describe* process*, not the live state of any specific org, so a few minutes of staleness rarely changes correctness. (2) live org metadata staleness — a user just changed a sharing rule and immediately asks the agent to act based on 'current' rules; here staleness can produce a genuinely wrong plan. The architecture's own answer to this is that* `TenantGrounding` *and live data queries during planning hit the live org directly, not the pre-ingested vector index — the vector index is for* documents *(NLDs, product docs), not live record state, so the specific failure mode of 'agent acts on stale org config' is mitigated by design: config-sensitive grounding is a live query, not a cached one — this is exactly Section 2.7's 'durable truth lives in the system of record, not in the model or its memory' principle, applied to the index/live-org split specifically. Where staleness* can *still bite is if an NLD's own content (not live org state) assumes something about a capability that just changed — and that's a content-freshness problem the human-curation step in the NLD pipeline is the actual control for, not a technical consistency guarantee.*" This is a strong answer because it doesn't just recite CAP vocabulary — it correctly separates two different data types with two different staleness costs and points to the actual architectural mitigation for the more dangerous one.

4.2 Idempotency and retries at the Executor boundary

Every Generic CRUD action call has to be safely retryable — a network timeout on `ExecuteEntityCreate` must not silently double-create a record on retry. Section 2.13 has the general idempotent-execution-wrapper pseudocode; here's the applied version:

- A client-generated idempotency key per logical operation (not per HTTP attempt) — typically derived from something like `(thread/plan ID, stepId)`, since a given plan step should only ever produce one real effect no matter how many times its underlying call is retried.
- The write layer dedups on that key before performing the actual create/update — a retry with the same key returns the *original* result rather than re-executing.
- Paired with **read-after-write verification** (pipeline stage 7): even if you trust the idempotency mechanism, you confirm the record actually exists in the expected state afterward rather than trusting the write call's 200 status alone — this catches the case where the write itself partially succeeded (e.g., the User record was created but a downstream PermissionSetAssignment in the same logical operation wasn't).
- **The harder version he might push toward:** "*What if the idempotency key's TTL expires and the same logical retry now looks like a brand-new request?*" Honest answer: idempotency keys need a retention window long enough to outlast any realistic retry/backoff sequence for that operation type — sized around the actual bounded retry policy's total time budget, not an arbitrary short TTL. If you don't know the exact number Salesforce uses here, say that plainly and reason about the tradeoff instead: too short a window reopens the double-create risk on a slow retry; too long a window means storing dedup state indefinitely for every write, which is its own storage-growth cost.

4.3 Rate limiting and backpressure — governor limits as the real-world analog

Salesforce's own per-org **governor limits** (SOQL query rows, DML statements, CPU time, API call counts, all per-transaction and per-24h-window) are effectively a platform-enforced rate limiter the Executor has to operate *inside of*, not just around — this is a great angle if he asks a generic rate-limiting question, because you can answer it against a real constraint instead of an abstract one. It's also the concrete instance of Section 2.11's "backpressure is a security control as much as a performance one."

How you'd reason about backpressure here: a multi-step `executionPlan` that fires several Generic CRUD calls (especially under the new parallel-execution proposal, 3.4) risks tripping a governor limit mid-plan if it doesn't budget for it. The right design mirrors a token-bucket rate limiter (`../CrowdStrike_Prep_Deep_Dive.md` §3.3 has the algorithm from scratch) but scoped **per org, per transaction window**, not globally — the Executor needs to know its remaining budget before fanning out a parallel stage, and degrade gracefully (fall back to serial execution, or defer non-critical read steps) rather than firing all N calls and letting some fail with a governor-limit exception mid-stage. **The failure-handling question that follows:** a governor-limit exception mid-plan is a *partial failure* with the same shape as 3.4's parallel-write problem — the steps that already succeeded are real side effects that happened; the Executor's job is to surface exactly what completed and what didn't, not to pretend the whole plan atomically failed when it didn't.

4.4 Delivery semantics for the extraction/ingestion pipeline

The Metadata Extraction BT Action's own state machine (4.1) is effectively an event-driven pipeline moving a document through processing stages — worth reasoning about in **at-least-once vs exactly-once** terms if he goes there generically. A reasonable, defensible position: this kind of pipeline should be **at-least-once with idempotent re-processing** (a document safely re-ingested from any stage produces the same final state, so a mid-pipeline retry or restart is harmless) rather than trying to guarantee exactly-once end-to-end — exactly-once across a multi-stage async pipeline is notoriously expensive to actually guarantee, and idempotent re-processing gets you the same practical safety without that cost.

4.5 Sharding / partitioning the knowledge index at scale

Tie this directly back to 3.2's disclosed scaling risk. The general trade-off (this is the applied version of consistent hashing / partitioning from the Deep Dive doc):

- **Shared index + tenant filter at query time** — simplest to build, but query cost scales with *total* corpus size across all tenants even though any one query only needs one tenant's slice; also the exact post-filter-vs-pre-filter leakage risk from Section 2.5.7/5.2 if the tenant filter isn't applied *before* the vector search itself.
- **Per-tenant partition (shard)** — a query only ever touches its own tenant's data, so query cost stays flat as *other* tenants' data grows; the cost moves to partition-management overhead (provisioning a new shard per tenant, rebalancing if tenant sizes are wildly uneven, more moving parts operationally). This is the consistent-hashing-style trade-off in miniature: partitioning trades query-time simplicity for operational complexity, and it's the right trade when per-tenant isolation is already a hard requirement anyway (which, here, it is — Section 5) — meaning the "extra" operational cost of partitioning is partially already being paid for the isolation guarantee, so the marginal cost of also getting a scaling benefit from it is lower than building both a filter-based isolation layer *and* a separate scaling fix.
- **What's actually described in the real design** leans toward the shared-index direction (UKD Dataspace shared across content teams; NLD index built from per-org-ingested mDLOs but the query-time architecture for how tenant-scoping is enforced against that index isn't fully detailed in what's disclosed here) — **say that gap honestly** if pushed for the exact current implementation, and pivot to the trade-off reasoning above as what you'd want to verify before proposing a specific fix.

4.6 Horizontal scaling of the Supervisor/Specialist nodes themselves

A genuinely hard question he might ask: "*Is the ConversationManager stateful per conversation — does that mean it's pinned to one process/instance, like a sticky session — or can any instance pick up any conversation turn?*" Reason about it out loud rather than guessing a specific implementation: the CM "owns the master state" and conversation continuity implies **some** durable state per conversation (thread) has to persist between turns — the architecturally clean way to get horizontal scalability *without* sticky sessions is exactly the durable-interrupt pattern from pipeline stage 5: state checkpointed to a real store keyed by conversation/thread ID, so *any* stateless compute instance can pick up the next turn by loading that state fresh, rather than requiring the same process to still be alive. This is Section 2.7's "checkpoint/proposal state" row, generalized. (This is also the identical pattern ResolveFlow implements concretely and verifiably — companion doc Section 1.5 — which is a strong bridge line here: "*I can't confirm that's exactly how the ConversationManager's state is persisted, but I've built and can show you the identical pattern end-to-end: LangGraph's checkpointer persists full state to Postgres keyed by thread ID specifically so a paused conversation can resume on a completely different process, potentially hours later — that's the general solution to 'stateful conversation, stateless horizontal scaling' whenever it comes up.*") The compute-heavy `compilePlan` call is comparatively easier to scale horizontally since it's closer to stateless-per-invocation (given its inputs) — the harder scaling problem is state continuity across turns, not raw compute parallelism.

---

5. Security — Salesforce first, ResolveFlow as supporting proof

Section 2.5 above is the full vendor-neutral security model (threat framing, confused deputy, prompt injection, hard vs. soft guardrails, secure tool design, the execution gateway pattern, HITL risk tiering). This section is where those principles show up specifically in Agentforce v2.

5.1 Tenant isolation — the orgJWT token-exchange pattern, deep

Already covered mechanically in 1.4 — the answer to have ready in one clean paragraph: "*Tenant isolation here isn't an application-level check that filters results after the fact — it's structural. A custom TokenExchange handler mints a scoped `orgJWT` per tenant before any retrieval call happens, and every downstream hop (the SFAP AI Gateway, the Data Cloud retrievers) trusts that token's scope rather than re-deriving authorization itself. That means the retrieval service literally never receives a credential that grants access to another org's data — cross-tenant leakage isn't 'prevented by an if-statement that could have a bug,' it's impossible given the token the service was handed, which is the stronger of the two guarantees.*" This is exactly Section 2.6's audience-bound-token principle, and exactly Section 2.5.3's strongest confused-deputy mitigation (execute under the caller's own scoped authority).

5.2 ACL-before-retrieval — why the ordering matters, not just the check

Same principle as `RAG_SIMPLE.md` §7 / `RESOLVEFLOW_DEEP_DIVE.md` §4.3 / Section 2.13's `acl_aware_search` pseudocode above, applied here: live-grounding queries against org state are **ACL-filtered before retrieval, not after** — the design states this explicitly. Restate the "why post-filter is a real leak" reasoning if pushed: restricted content that enters the candidate set before a filter runs has already been read by anything downstream (a summarizer, a reranker) even if the final filter blocks it from the user; a caller getting fewer results than usual leaks the *existence* of restricted content through the shortfall itself; and pre-filtering (scoping the query/token itself, as in 5.1) makes leakage impossible by construction rather than merely prevented by application code that has to get it right every single time.

5.3 Confused deputy, avoided the same way twice

State the general pattern once (Section 2.5.3), then the two instances: "*The structural fix for 'confused deputy' — an agent tricked into using its own broad permissions on an attacker's behalf — is that authorization is derived from the actual caller's identity and grants, never from a shared powerful credential the agent holds on everyone's behalf.*

Here that's the `orgJWT` minted per-tenant and the operation-level allowlist checked at the Executor against the requesting user's actual permission set — the model proposes, code disposes, and the credential used for the actual write was never broader than what that specific caller already had. I've built the identical pattern where I can show full code: ResolveFlow's write function runs under the requesting visitor's *own OAuth token, never a shared service credential, so GitHub's own permission model does real enforcement — a visitor can comment but can't necessarily label, depending on their own actual access to that repo.*"

5.4 If he wants exhaustive, code-level security depth

Say: "*The Salesforce implementation I can speak to at the pattern level; for exact code-level defense-in-depth — including a couple of real vulnerabilities I found and fixed, like a broken-object-level-authorization bug and a cost-abuse vector — I have a public project with the full source. Want me to walk through one of those in complete depth?*" Then pull from `RESOLVEFLOW_DEEP_DIVE.md` Section 2 (the six real, fixed security bugs) and Section 3 (the red-team table) — don't try to recall them from memory here, that doc has the precise mechanism for each.

---

6. System design & scaling prompts — Salesforce-flavored, ready to whiteboard

Use the framework from the Hiren doc Part F, restated: clarify users/scale/risk/latency → define trust boundaries early → architecture + request flow → deep-dive → eval/ rollout → cost/latency/consistency trade-offs → phased, measurable MVP.

1. "Design the Agentforce for Setup v2 architecture from scratch" — you have this cold now: pipeline (1.2) → three pillars (1.3–1.5) → funnel for the scale story (1.3) → migration strategy (co-existence → content migration → action consolidation) → the disclosed risks (3.2–3.4) as the honest trade-offs section.

2. "Design the Parallel Action Execution feature" — this is a real, live design problem from the actual roadmap (3.4). Lead with the dependency-graph analysis at compile time, the fan-out/join Executor mechanics, and be ready to go deep on partial failure — that's the part he's most likely to push on, and "reads are safely partial, writes need a saga/compensation story I'd want to design carefully" is a strong, honest answer.

3. "Design a rate limiter for agent actions against a resource with hard governor limits" — token-bucket, scoped per-org-per-window (4.3), with graceful degradation (fall back to serial, defer non-critical steps) rather than partial-plan failure as the default behavior when budget is tight.

4. "Design the tenant-scoped retrieval system for the NLD/metadata index at 500K-org scale" — this is 3.2 + 4.5 combined: state the shared-index-with-filter vs per-tenant-shard trade-off, note that isolation is already a hard requirement here (5.1) so partitioning's operational cost is partly already justified by that, not just by scale.

5. "Design the index-freshness reconciliation for the mDLO pipeline" — apply 4.1: distinguish NLD-content staleness (low-stakes, human-curated so infrequent, tolerable lag) from live-org-state staleness (higher-stakes, which is why live grounding queries bypass the async pipeline entirely rather than relying on it) — the reconciliation design differs for each.

6. "How would you evaluate whether this platform is trustworthy enough to expand from pilot to all of Setup?" — golden-set task success rate threshold per workflow, shadow-mode period for new capabilities before real traffic, human-override rate as a trust signal (a high override rate on approvals means low real-world confidence regardless of what confidence scoring says), incident/rollback plan per workflow (not one big rollback switch), phased rollout gated per workflow the same way the V1→V2 migration itself was gated (Story B).

7. "You're signing off on an autonomous, revenue-affecting agent — what are your non-negotiables?" (Section 2.15's security-architect-tier question, verbatim) — answer with Section 2.14's 22 lessons as your

checklist, land on Section 2.16's litmus test as the final gate: bounded, small, attributable, reversible blast radius, or it doesn't ship.

---

7. ResolveFlow — proof of mechanism, not the headline

One paragraph, then bridge, not a full re-derivation:

> "ResolveFlow is a portfolio project — a LangGraph agent that diagnoses GitHub issues and can propose a fix, where reasoning and execution are separated by construction: two independent LLM calls have to agree a diagnosis is grounded, a human approves the exact text before anything posts, and the one function allowed to write to GitHub refuses to run without an explicit approval flag checked in the function itself, not just at the routing layer. It's the same skeleton as Agentforce for Setup v2 — retrieve grounding evidence, compile a deterministic plan, gate risky writes behind human approval, execute with no reasoning in the write path, verify after — applied to a different domain where I can show you the actual code."

When to reach for it: only when (a) he explicitly asks for code-level depth Salesforce confidentiality won't let you give, or (b) a security question wants the kind of concrete, verifiable detail only a fully-inspectable system provides — real thresholds, a real fixed IDOR bug, an exact checkpoint schema. The full depth — architecture walkthrough, the independent-review pattern, the interrupt/checkpoint mechanism, six real fixed security bugs from the actual commit history, a red-team table, and a whiteboard sequence diagram — is entirely in `RESOLVEFLOW_DEEP_DIVE.md`. Don't try to recall it from memory in the room; if he goes deep here, it's fine to say "let me walk through this precisely" and draw from what you know is in that doc.

---

8. The 45 minutes, Salesforce-led

- **Open with the Section 1.1 pitch** when he asks "walk me through what you build" — then stop and let him choose where to go deep, rather than front-loading a monologue.
- **Wherever he goes, answer at pipeline-stage precision** (1.2) before he has to ask "which stage" — naming the stage is itself a signal you think in this architecture's own terms, not an improvised one.
- **If the question is generic rather than Agentforce-specific, answer from Section 2 first, then instantiate it in Section 1's terms** — that back-and-forth fluency is the single strongest signal of real depth versus a memorized system.
- **If he goes to scale**, you now have real, specific material (Section 3) instead of generic "it scales horizontally" — use the funnel, the parallel-execution schema, and the streaming-UX reasoning by name.
- **If he goes distributed-systems-generic** (CAP, rate limiting, sharding, consistency, delivery semantics), Section 4 has the applied answer for each, tied to a real component of this system — always land back on the specific mechanism, not just the vocabulary.
- **Volunteer one disclosed limitation early** — the centralized-retrieval scaling risk (3.2) or the AgentGraph-vs-ReAct latency regression (both straight from the design's own "Risks" section) are safe, honest, and specific.
- **Bridge to ResolveFlow once, deliberately, when it earns its place** — not as a second opening.
- **If he tries to catch you overclaiming, run Section 2.16's litmus test out loud** — it's the fastest, most credible way to self-correct in real time.
- **Closing questions** (adapted from the Hiren doc, sharpened for a security-company, fiercely-technical interviewer): "*Where has agentic AI actually failed in production here, in a way that changed the architecture — not hypothetically?*" / "*What's the security review bar for a new action being added to the allowlist — who signs off, and what would block it?*" / "*At CrowdStrike's scale and threat model specifically, where do you expect agent tenant-isolation to be tested hardest — is it more a retrieval problem or an execution problem, in your experience?*"

Cross-references: `INTERVIEW_PREP_Hiren_Anajwala_Architecture.md` *for Parts C (MCP/integration deep dive), G (rapid-fire architecture Q&A), and H (questions to ask);* `../CrowdStrike_Prep_Deep_Dive.md` *§3 for distributed-systems mechanisms derived from scratch;* `RESOLVEFLOW_DEEP_DIVE.md` *for full ResolveFlow code-level depth;* `../INTERVIEW_PREP_CrowdStrike_LeadAIEngineer_GTM.md` *for fit/gap and "why CrowdStrike" if this round touches motivation;* `study_guide_enterprise_system.md` *is the original source for Section 2 — re-read it directly if you want the material in its own voice rather than woven into this doc's cross-references.*
